HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN

BỘ KHOA HỌC VÀ CÔNG NGHỆ



PROJECT 01

HỆ ĐA AGENT CHO BỆNH TIỂU ĐƯỜNG

Giảng viên hướng dẫn : **PGS.TS Trần Đình Quế**

Họ và tên sinh viên : **Hoàng Hải**

Mã sinh viên : **B23DCCN270**

Lớp nhóm : **05**

Hà Nội – 2026

Mục lục

Thông tin project 3
CHƯƠNG 1 GIỚI THIỆU 4
1.1 Bối cảnh và bài toán 4
1.2 Yêu cầu từ sơ đồ trên bảng 4
1.3 Mục tiêu và phạm vi 6
1.4 Môi trường thực hiện 6
CHƯƠNG 2 HỆ ĐA AGENT 8
2.1 Agent 8
2.2 Hệ đa agent 8
2.3 Điều phối và giao tiếp 8
2.4 Học tăng cường 8
2.5 Kết hợp RL và Rule 9
CHƯƠNG 3 KIẾN TRÚC TỔNG QUAN 10
3.1 Sơ đồ kiến trúc 10
3.2 Vai trò các agent 10
3.3 Luồng xử lý 11
3.4 Các bất biến an toàn 11
CHƯƠNG 4 CÁC THÀNH PHẦN HỆ THỐNG 12
4.1 Deep Learning ⇒ Agent 12
4.2 KB + RAG 17
4.3 Reinforcement Learning + Rule 23
4.4 Các agent tư vấn, trợ giúp và điều phối 30
CHƯƠNG 5 CODE VÀ DEMO 39
5.1 Tổ chức notebook và cách chạy 39
5.2 Demo các truy vấn 40
5.3 Kiểm thử và tiêm lỗi 49
5.4 Đánh giá toàn hệ thống 51
5.5 Phân tích lỗi 54
KẾT LUẬN 55
Kết quả đạt được 55
Giới hạn 55
Hướng phát triển 55
TÀI LIỆU THAM KHẢO 56

Thông tin project

Đề tài là **hệ đa agent cho bệnh tiểu đường**, hiện thực sơ đồ giảng viên vẽ trên bảng: dữ liệu có sẵn đi qua Deep Learning để dự đoán và phân loại; web và sách tạo thành KB; KB kết hợp RAG và Agent; Agent được xây từ Reinforcement Learning và Rule; các nhánh hợp lại để tư vấn và trợ giúp người dùng. Báo cáo theo dàn ý trên bảng gồm giới thiệu, hệ đa agent, kiến trúc tổng quan, các thành phần hệ thống, code và demo.

Sản phẩm nộp

Tệp	Nội dung
project01_05_15_haih.ipynb	Toàn bộ mã nguồn và output đã chạy, dùng để trình bày
project01_05_15_haih.docx	Báo cáo này
data/	Hai bộ dữ liệu, kho KB, bộ câu hỏi và kịch bản đánh giá
results/	Mô hình, bảng số liệu, hình và ảnh chụp từng cell
README.md, start_notebook.bat	Hướng dẫn và lệnh mở notebook

Mọi số liệu trong báo cáo lấy từ output của notebook đã chạy trên máy cá nhân chỉ dùng CPU. Hệ thống phục vụ học tập, không chẩn đoán và không thay thế bác sĩ. Bố cục trình bày có tham khảo bài Project của bạn Phan Văn Khôi [21]; mã nguồn, dữ liệu KB, thí nghiệm và số liệu ở đây được xây dựng và chạy riêng. Điểm khác chính là có nhánh Reinforcement Learning + Rule như trên bảng.

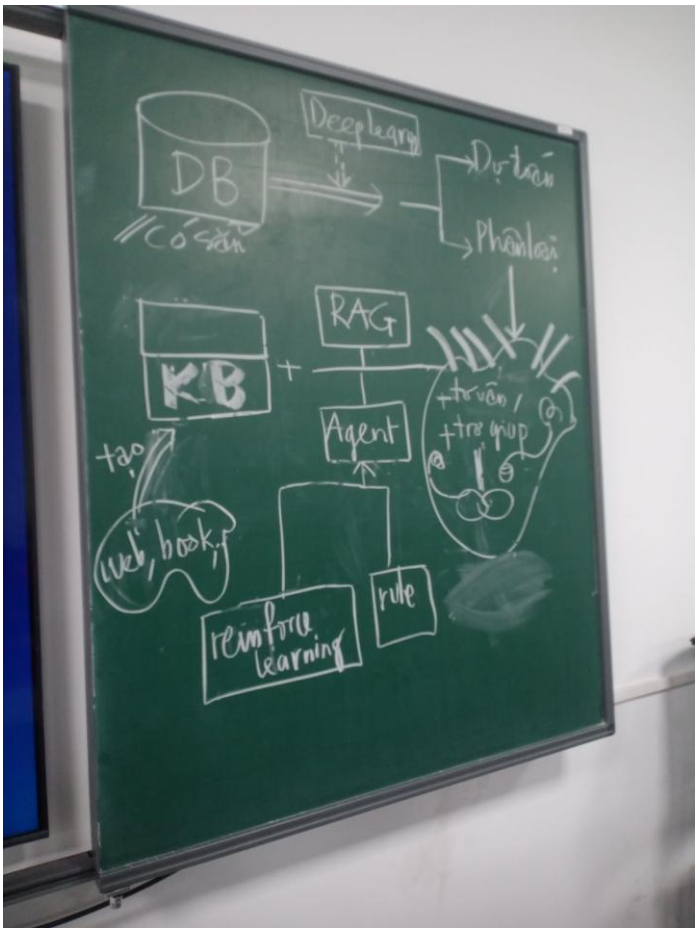
CHƯƠNG 1 GIỚI THIỆU

1.1 Bối cảnh và bài toán

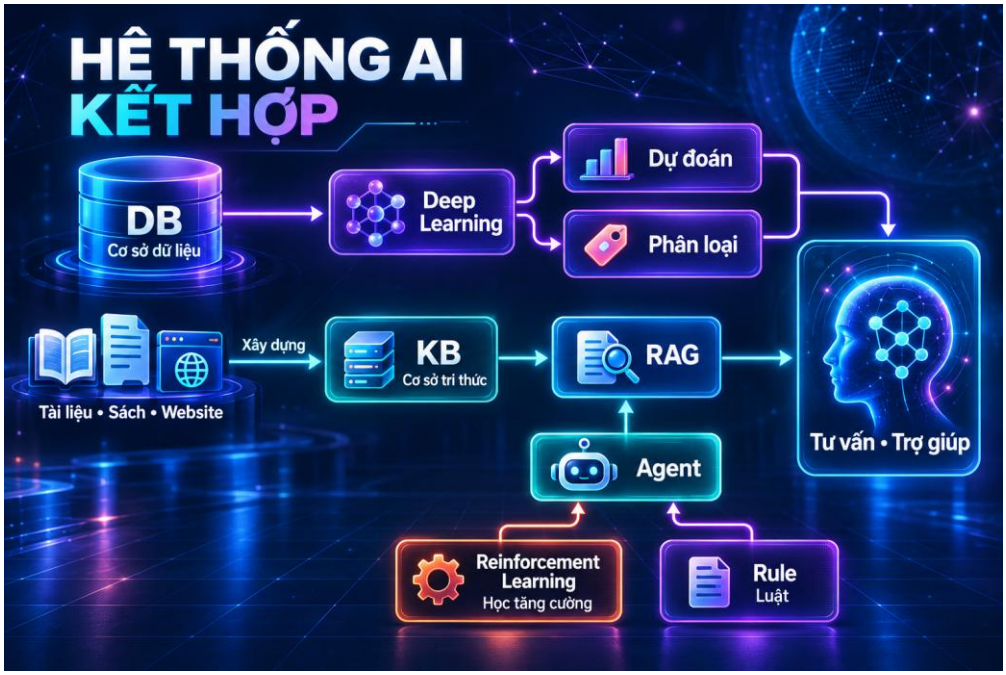
Đái tháo đường, thường gọi là tiểu đường, là bệnh mạn tính phổ biến. WHO cho biết số người sống với bệnh tăng từ 200 triệu năm 1990 lên 830 triệu năm 2022 [16]. Type 2 có thể tiến triển âm thầm nhiều năm, vì vậy việc phát hiện dựa vào khám và xét nghiệm định kỳ hơn là chờ triệu chứng [10][16].

Một mô hình học máy đơn lẻ chỉ trả về xác suất. Xác suất đó chưa trả lời được người dùng nên làm gì tiếp theo, vì sao có kết quả như vậy, nguồn nào xác nhận và khi nào phải đi khám ngay. Bài toán của đồ án vì thế không dừng ở phân loại mà là xây dựng một hệ thống biết dự đoán, tra cứu tri thức, ra quyết định và hỗ trợ người dùng trong cùng một luồng.

1.2 Yêu cầu từ sơ đồ trên bảng



Hình 1.1 Sơ đồ giảng viên vẽ trên bảng, là yêu cầu trực tiếp của project.



Hình 1.2 Sơ đồ trên bảng được vẽ lại rõ hơn.

Bảng ghi hai yêu cầu: sinh ra hệ thống theo hình trên bảng và tiếp tục phát triển đầy đủ như hình. Bảng 1.2 nêu cách từng khối được hiện thực.

Bảng 1.1 Dàn ý trên bảng và vị trí trong báo cáo

Mục trên bảng	Vị trí
1. Giới thiệu	Chương 1
2. Hệ đa agent, 2.1 Agent	Chương 2
3. Kiến trúc tổng quan	Chương 3
4. Các thành phần hệ thống: 4.1 Deep Learning ⇒ Agent, 4.2 KB + RAG, 4.…	Chương 4, thêm 4.3 RL + Rule và 4.4 các agent tư vấn, trợ giúp
5. Code + Demo	Chương 5

Bảng 1.2 Ánh xạ khối trên bảng sang hệ thống

Khối trên bảng	Hiện thực trong đồ án
DB có sẵn	diabetes_100k.csv (100.000 dòng) và diabetes_risk_50k.csv (50.000 dòng)
Deep Learning	DiaNet và RiskNet viết bằng PyTorch
Phân loại	DiaNet: xác suất mắc và nhãn; RiskNet: mức Low, Moderate, High
Dự đoán	RiskNet: điểm nguy cơ 0–100 và mô phỏng thay đổi
web, book → tạo KB	78 ghi chú có nguồn từ NIDDK, WHO, Diabetes UK và một cuốn sách hướng dẫn của HHS
KB	Knowledge Graph 156 thực thể, 183 quan hệ, bảng 11 ngưỡng
RAG	Truy xuất lai lexical, vector, KG; xếp hạng RRF
Agent	Coordinator và 7 agent chuyên trách
Reinforcement Learning	Q-learning cho Policy Agent
Rule	Ba luật R1, R2, R3 của Policy Agent và các luật của Safety Agent
Tư vấn, trợ giúp	Advisor Agent và Support Agent

1.3 Mục tiêu và phạm vi

- Huấn luyện mô hình Deep Learning xử lý được hồ sơ thiếu xét nghiệm.
- Xây KB có nguồn rõ ràng, truy xuất được và kiểm tra được bằng code.
- Dùng RL để chọn bước tiếp theo, dùng Rule để giữ các ràng buộc an toàn.
- Trả lời bằng tiếng Việt, có trích dẫn, không nêu liều thuốc, không khẳng định người dùng mắc bệnh.
- Đánh giá bằng số đo trên tập test, so với mốc đơn giản hơn, có kiểm thử lỗi.

Theo bài giảng về hệ đa agent, nguyên mẫu sinh viên trong lĩnh vực y tế nên giới hạn ở hỗ trợ giáo dục và chuẩn bị đi khám, không tự chẩn đoán hay điều trị [20]. Đồ án giữ đúng ranh giới này.

1.4 Môi trường thực hiện

Bảng 1.3 Công nghệ sử dụng

Thành phần	Lựa chọn
Phần cứng	Máy Windows 11, 8 GB RAM, không dùng GPU
Ngôn ngữ	Python 3.13, Jupyter Notebook
Deep Learning	PyTorch 2.15 (CPU) [19]
Mốc so sánh học máy	scikit-learn 1.9: Logistic Regression, HistGradientBoosting
Knowledge Graph	NetworkX; xuất kèm tệp Cypher cho Neo4j
RAG	TF-IDF, LSA, RRF tự cài đặt trên scikit-learn
RL	Q-learning dạng bảng tự cài đặt bằng NumPy

Bài giảng gợi ý Neo4j, Sentence Transformers và LLM [20]. Ba công cụ này chưa dùng được trong đồ án. Máy chưa cài Neo4j và không có LLM cục bộ; model embedding Transformer cũng không

nạp được do thiếu bộ nhớ. Đồ án thay bằng NetworkX, vector LSA và câu trả lời ghép từ mẫu câu cùng nguyên văn ghi chú. Hệ quả được phân tích ở phần giới hạn.

CHƯƠNG 2 HỆ ĐA AGENT

2.1 Agent

Agent là thực thể tính toán nhận thông tin từ môi trường và chọn hành động để đạt mục tiêu [1][20]. Tại thời điểm t, agent nhận quan sát o, có thể giữ trạng thái nội tại b và chọn hành động theo chính sách a = π(b, o). Agent khác một hàm thông thường ở chỗ nó có mục tiêu, có chính sách quyết định và phản ứng theo tình huống.

Đặc tả PEAS cho hệ thống được nêu trong Bảng 2.1. Môi trường ở đây quan sát được một phần vì hồ sơ thường thiếu, có tính ngẫu nhiên vì kết quả xét nghiệm chưa biết trước và có tính tuần tự vì mỗi lượt hỏi đổi thông tin đã có.

Bảng 2.1 Đặc tả PEAS

Yếu tố	Mô tả
Performance	Đúng tuyến xử lý, ít bỏ sót, ít xét nghiệm thừa, có trích dẫn, không vi phạm an toàn
Environment	Câu hỏi tiếng Việt, hồ sơ thiếu thông tin, KB, các mô hình đã huấn luyện
Actuators	Hỏi lại, yêu cầu xét nghiệm, tư vấn, chuyển khám, cảnh báo khẩn cấp, lập danh sách chuẩn bị
Sensors	Tin nhắn người dùng, bộ nhớ phiên, xác suất từ DL, bằng chứng từ RAG

2.2 Hệ đa agent

Hệ đa agent gồm nhiều agent có trách nhiệm riêng, tương tác qua thông điệp hoặc trạng thái chung [2]. Bài giảng lưu ý rằng một tập hàm được đổi tên thành agent chưa phải hệ đa agent: ranh giới agent phải ứng với trách nhiệm, chính sách hoặc công cụ khác nhau [20]. Trong đồ án, mỗi agent có chính sách riêng: Prediction dùng mạng nơ ron, Policy dùng RL và Rule, Knowledge dùng truy xuất, Safety dùng luật kiểm duyệt.

Bài toán tiểu đường phù hợp với cách chia này vì các phần có bản chất khác nhau. Xác suất cần học từ dữ liệu. Ngưỡng chẩn đoán cần lấy từ tài liệu. Quyết định xét nghiệm cần cân đối chi phí và rủi ro. Tình huống khẩn cấp cần luật chắc chắn. Gộp cả vào một khối sẽ khó kiểm tra phần nào sai.

2.3 Điều phối và giao tiếp

Đồ án dùng điều phối tập trung. Coordinator gửi yêu cầu, nhận kết quả, kiểm tra schema và chọn bước tiếp theo. Cách này dễ theo dõi và dễ kiểm chứng đầu ra, đổi lại Coordinator là điểm phụ thuộc chung [20]. Thông điệp có người gửi, người nhận, kiểu, nội dung, mã hội thoại và thời điểm. Agent trả sai schema bị coi là lỗi, Coordinator thử lại một lần rồi dùng phương án dự phòng.

2.4 Học tăng cường

Học tăng cường tìm chính sách tối đa tổng phần thưởng trong quá trình quyết định Markov [3]. Q-learning cập nhật giá trị hành động theo công thức:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \, [\, r + \gamma \, \max_{a'} Q(s', a') - Q(s, a) \,]$$

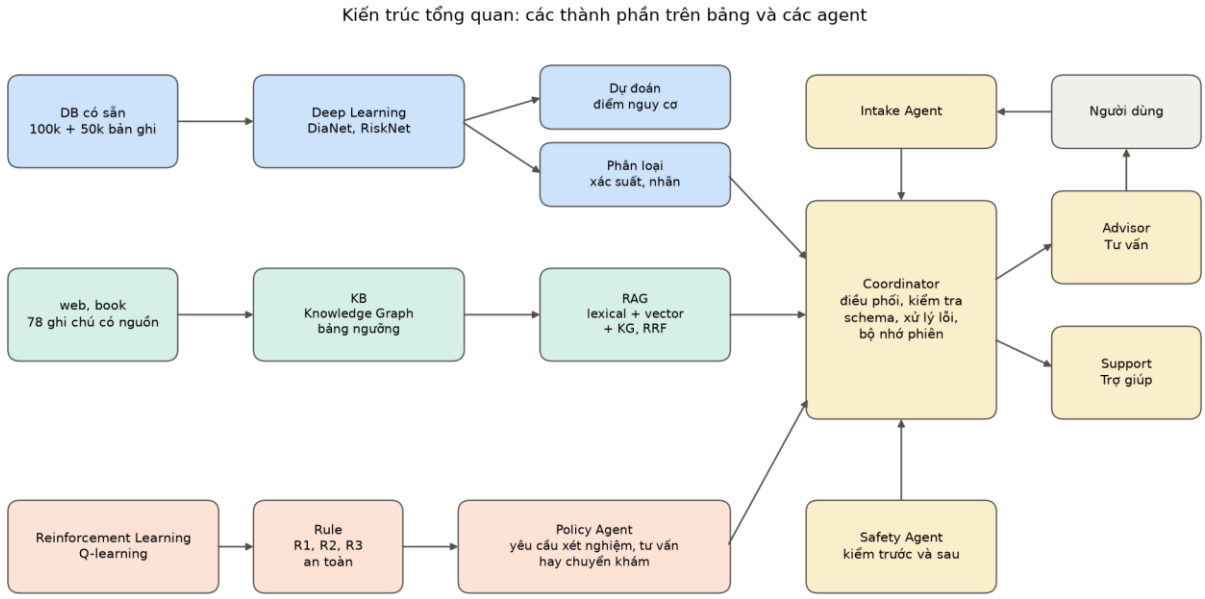
Trong đó s là trạng thái, a là hành động, r là phần thưởng, α là tốc độ học và γ là hệ số chiết khấu [8]. Thuật toán không cần biết trước mô hình chuyển trạng thái. Chính sách ε-greedy cân bằng giữa khám phá và khai thác.

2.5 Kết hợp RL và Rule

RL tối ưu phần thưởng trung bình nên có thể chấp nhận sai ở những ca hiếm. Rule không học nhưng bảo đảm ràng buộc cứng. Trên bảng, Agent nhận cả hai đầu vào. Đồ án hiện thực theo dạng shield: RL đề xuất hành động, Rule ghi đè khi đề xuất vi phạm ràng buộc [9]. Bài giảng cũng khuyến nghị kiến trúc lai, trong đó luật giữ ràng buộc an toàn, mô hình học xếp hạng hoặc phân loại, Coordinator quản lý luồng và con người chịu trách nhiệm quyết định quan trọng [20].

CHƯƠNG 3 KIẾN TRÚC TỔNG QUAN

3.1 Sơ đồ kiến trúc



Hình 3.1 Kiến trúc tổng quan của hệ thống.

Ba nhánh bên trái tương ứng ba nhánh trên bảng. Nhánh dữ liệu cho xác suất và điểm nguy cơ. Nhánh tri thức cho bằng chứng có nguồn. Nhánh quyết định cho hành động tiếp theo. Coordinator nối ba nhánh, Intake nhận đầu vào, Safety kiểm tra trước và sau, Advisor và Support tạo đầu ra.

3.2 Vai trò các agent

Bảng 3.1 Đặc tả agent

Agent	Đầu vào	Đầu ra	Công cụ	Chính sách
Coordinator	Mọi thông điệp	Kết quả cuối, trace	Bộ nhớ phiên	Luồng theo trạng thái, retry, fallback
Intake	Văn bản, bộ nhớ	Hồ sơ, ý định, lỗi	parse_profile	Luật regex tiếng Việt
Safety	Văn bản, bản nháp	Mức an toàn, lỗi	red_flags, critic	Luật
Prediction	Hồ sơ	Xác suất, nhãn, điểm	DiaNet, RiskNet	Deep Learning
Policy	Xác suất, hồ sơ	Hành động, luật đã dùng	Q-table, threshold_lookup	RL + Rule
Knowledge	Truy vấn	Bằng chứng, quan hệ KG	search_kb, kg_neighbors	RAG, tự mở rộng truy vấn
Advisor	Kết quả các agent	Câu trả lời có [n]	compose	Mẫu câu bám bằng chứng
Support	Hồ sơ, bằng chứng	Danh sách chuẩn bị	make_checklist	Mẫu câu bám bằng chứng

3.3 Luồng xử lý

Luồng không cố định. Sau khi Intake và Safety chạy, Coordinator rẽ nhánh theo kết quả:

- **Khẩn cấp:** bỏ qua mô hình, lấy tài liệu xử trí và yêu cầu gọi cấp cứu.
- **Ranh giới thuốc:** từ chối kê đơn, chỉ cung cấp thông tin chung.
- **Thiếu hoặc sai dữ liệu:** hỏi lại, không chạy mô hình, ghi nhớ yêu cầu đang dở.
- **Đánh giá nguy cơ:** Prediction → Policy → Knowledge, thêm Support khi cần chuyển khám.
- **Hỏi kiến thức:** chỉ qua Knowledge.
- **Trợ giúp:** Knowledge → Support.
- **Ngoài phạm vi:** từ chối lịch sự, không truy xuất.

Sau khi có bản nháp, Safety kiểm duyệt. Nếu thiếu bằng chứng cho một ngưỡng, Coordinator quay lại Knowledge rồi Advisor viết lại, tối đa hai vòng. Nếu vẫn không đạt, hệ thống chặn nội dung chi tiết. Trên 40 kịch bản, cấu hình đầy đủ đi qua 7 đường khác nhau (Bảng 3.2), cho thấy luồng được quyết định lúc chạy.

Bảng 3.2 Các đường đi thực tế của cấu hình đầy đủ

Đường đi	Số kịch bản
intake → safety → knowledge → advisor → safety	16
intake → safety → advisor → safety	7
intake → safety → prediction → policy → knowledge → advisor → safety	5
intake → safety → prediction → policy → knowledge → advisor → safety → knowledge → advisor → safety	4
intake → safety → prediction → policy → knowledge → knowledge → support → advisor → safety → knowledge → advisor → safety	3
intake → safety → knowledge → support → advisor → safety	3
intake → safety → prediction → policy → knowledge → knowledge → support → advisor → safety	2

3.4 Các bất biến an toàn

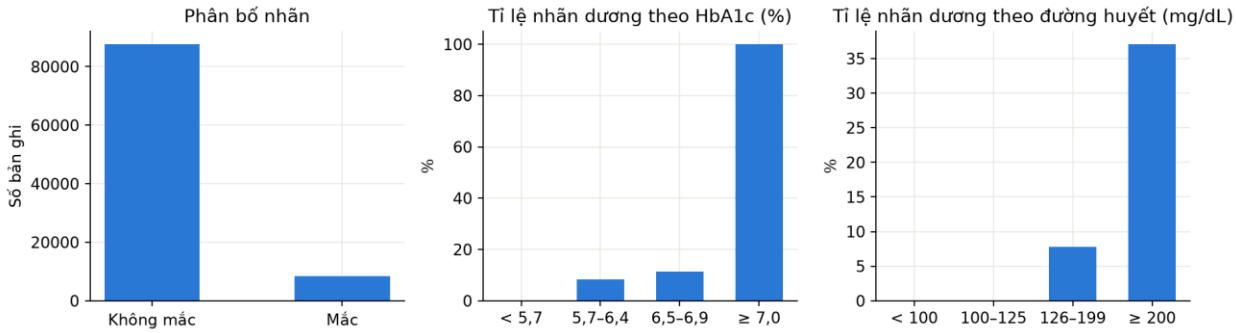
- Xác suất mô hình không được trình bày như chẩn đoán.
- Chỉ số đạt ngưỡng thì phải nêu ra và phải khuyên gặp bác sĩ, kể cả khi mô hình báo thấp.
- Không nêu liều thuốc, không hướng dẫn tự đổi thuốc.
- Mọi con số kèm %, mg/dL, mmol/L phải đến từ người dùng, mô hình hoặc nguồn.
- Không có bằng chứng thì nói chưa đủ thông tin.
- Hệ thống không tự đặt lịch hay thay đổi hồ sơ.

CHƯƠNG 4 CÁC THÀNH PHẦN HỆ THỐNG

4.1 Deep Learning ⇒Agent

4.1.1 Dữ liệu

Bộ 100.000 bản ghi có các cột giới tính, tuổi, tăng huyết áp, bệnh tim, hút thuốc, BMI, HbA1c, đường huyết và nhãn diabetes. Sau khi bỏ dòng trùng và 18 dòng gender là Other còn 96.128 dòng, tỉ lệ dương 8,82%. Dữ liệu chia phân tầng 70/15/15 thành 67.289 dòng train, 14.419 dòng validation và 14.420 dòng test. Bỏ trùng trước khi chia để cùng một bản ghi không lọt vào cả train và test.



Hình 4.1 Phân bố nhãn và tỉ lệ nhãn dương theo xét nghiệm.

Mâu thuẫn giữa dữ liệu và tri thức. Tài liệu y khoa xếp HbA1c từ 6,5% vào ngưỡng đái tháo đường [10], nhưng trong dữ liệu chỉ 11,5% số dòng HbA1c 6,5–6,9% có nhãn dương. Mô hình học từ dữ liệu này có thể báo nguy cơ thấp cho người đã đạt ngưỡng. Phát hiện này quyết định thiết kế: Rule phải độc lập với mô hình và được quyền ghi đè.

4.1.2 DiaNet: phân loại

Người dùng thường chưa có đủ xét nghiệm. Thay vì huấn luyện bốn mô hình, DiaNet nhận thêm hai bit mask cho biết HbA1c và đường huyết đã biết hay chưa. Khi huấn luyện, mỗi xét nghiệm bị che ngẫu nhiên với xác suất 0,5. Mạng có 15 đầu vào, ba lớp ẩn 128, 64, 32 với ReLU và Dropout 0,1, đầu ra sigmoid. Hàm mất mát là binary cross-entropy không gán trọng số lớp để giữ xác suất hiệu chỉnh tốt, vì Policy Agent dùng trực tiếp xác suất này. Ngưỡng phân loại chọn trên validation theo F1.

In [4]:

```python
class MaskedMLP(nn.Module):
    """MLP dùng chung cho DiaNet và RiskNet; mỗi đầu ra là một head."""
    def __init__(self, d_in, heads, hidden=(128, 64, 32), dropout=.1):
        super().__init__()
        layers, d = [], d_in
        for width in hidden:
            layers += [nn.Linear(d, width), nn.ReLU(), nn.Dropout(dropout)]
            d = width
        self.body = nn.Sequential(*layers)
        self.heads = nn.ModuleDict({name: nn.Linear(d, n) for name, n in heads.items()})

    def forward(self, x):
        h = self.body(x)
        return {name: head(h) for name, head in self.heads.items()}

SMOKE = ["never", "No Info", "current", "former", "ever", "not current"]
STATS = dict(age=(train_df.age.mean(), train_df.age.std()), bmi=(train_df.bmi.mean(), train_df.bmi.std()),
             h=(train_df.HbA1c_level.mean(), train_df.HbA1c_level.std()),
             g=(train_df.blood_glucose_level.mean(), train_df.blood_glucose_level.std()))
SCEN = ["none", "hba1c", "glucose", "both"]  # chỉ số = bit0 HbA1c + 2*bit1 glucose
SCEN_VI = {"none": "Chưa xét nghiệm", "hba1c": "Chỉ HbA1c", "glucose": "Chỉ đường huyết", "both": "Đủ hai xét nghiệm"}

def z(v, key):
    return ((np.asarray(v, dtype=np.float64) - STATS[key][0]) / STATS[key][1]).astype(np.float32)

def dia_arrays(d):
    """Trả về đặc trưng nền, HbA1c và đường huyết đã chuẩn hóa."""
    base = np.column_stack([z(d.age, "age"), z(d.bmi, "bmi"), d.hypertension, d.heart_disease, (d.gender == "Male") * 1.0]
                           + [(d.smoking_history == s) * 1.0 for s in SMOKE]).astype(np.float32)
    return base, z(d.HbA1c_level, "h"), z(d.blood_glucose_level, "g")

def dia_input(base, h, g, mh, mg):
    return np.column_stack([base, h * mh, g * mg, mh, mg]).astype(np.float32)

def predict_dia(model, arrays, scen):
    base, h, g = arrays
    k = SCEN.index(scen)
    mh, mg = np.full(len(h), k & 1, np.float32), np.full(len(h), k >> 1, np.float32)
    model.eval()
    with torch.no_grad():
        return torch.sigmoid(model(torch.tensor(dia_input(base, h, g, mh, mg)))["cls"]).numpy().ravel()

A_train, A_val, A_test = dia_arrays(train_df), dia_arrays(val_df), dia_arrays(test_df)
y_train, y_val, y_test = train_df.diabetes.values, val_df.diabetes.values, test_df.diabetes.values
print("Số đặc trưng đầu vào của DiaNet:", A_val[0].shape[1] + 4)
```

```
Số đặc trưng đầu vào của DiaNet: 15
```

Hình 4.2 Cell [4] định nghĩa MaskedMLP và đầu vào có mask của DiaNet.

In [5]:
```
def train_dianet(seed, epochs=30, batch=1024, patience=5, p_mask=.5):
    set_seed(seed)
    base, h, g = (torch.tensor(a) for a in A_train)
    y = torch.tensor(y_train, dtype=torch.float32)
    model = MaskedMLP(base.shape[1] + 4, {"cls": 1})
    opt = torch.optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-5)
    loss_fn = nn.BCEWithLogitsLoss()
    best, best_state, wait, history = -1, None, 0, []
    for epoch in range(epochs):
        model.train()
        perm, total = torch.randperm(len(y)), 0.
        for i in range(0, len(y), batch):
            b = perm[i:i + batch]
            mh = (torch.rand(len(b)) > p_mask).float()  # che ngẫu nhiên từng xét nghiệm
            mg = (torch.rand(len(b)) > p_mask).float()
            x = torch.cat([base[b], (h[b] * mh)[:, None], (g[b] * mg)[:, None], mh[:, None], mg[:, None]], 1)
            loss = loss_fn(model(x)["cls"].squeeze(1), y[b])
            opt.zero_grad(); loss.backward(); opt.step()
            total += loss.item() * len(b)
        val_ap = np.mean([average_precision_score(y_val, predict_dia(model, A_val, s)) for s in SCEN])
        history.append((epoch + 1, total / len(y), val_ap))
        if val_ap > best + 1e-4:
            best, wait = val_ap, 0
            best_state = {k: v.clone() for k, v in model.state_dict().items()}
        else:
            wait += 1
            if wait >= patience:
                break
    model.load_state_dict(best_state)
    return model, pd.DataFrame(history, columns=["epoch", "train_loss", "val_pr_auc"])

t0 = time.time()
runs = {s: train_dianet(s) for s in (42, 7, 2026)}
dianet, dia_hist = runs[42]
torch.save(dianet.state_dict(), RES / "dianet.pt")
print(f"Huấn luyện 3 seed trong {time.time() - t0:.0f} giây; seed 42 dừng ở epoch {len(dia_hist)}")
dia_hist.tail(3).round(4)
```

```
Huấn luyện 3 seed trong 50 giây; seed 42 dừng ở epoch 30
```

Out[5]:

	epoch	train_loss	val_pr_auc
27	28	0.1522	0.6625
28	29	0.1513	0.6623
29	30	0.1496	0.6629

Hình 4.3 Cell [5] huấn luyện DiaNet với mask ngẫu nhiên và dừng sớm theo PR-AUC validation.

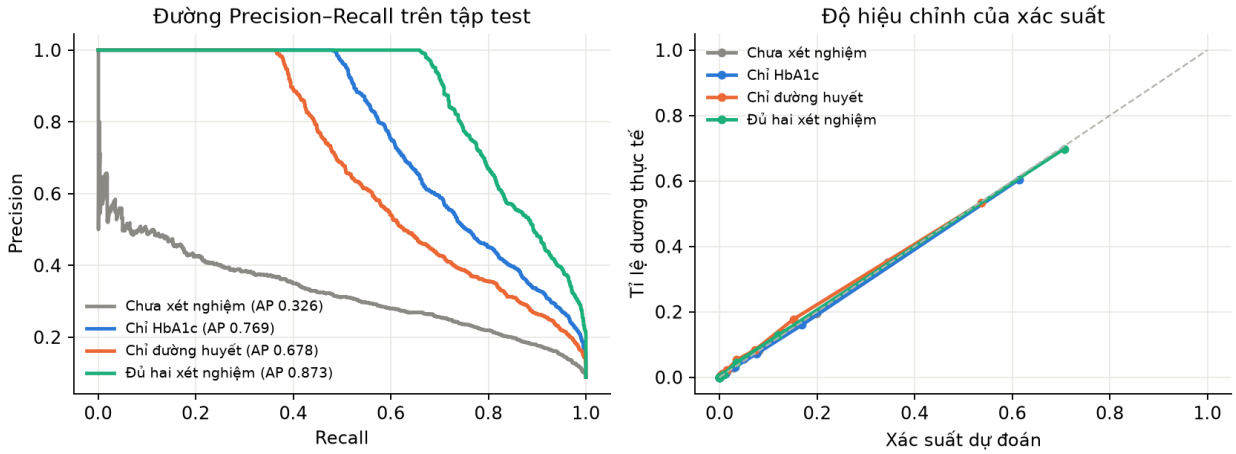
Bảng 4.1 Kết quả phân loại trên tập test (14.420 dòng)

Trường hợp	Mô hình	PR-AUC	ROC-AUC	F1	Precision	Recall	Brier	ECE
Chưa xét nghiệm	LR	0,315	0,832	0,381	0,293	0,544	0,069	0,006
Chưa xét nghiệm	HGB	0,322	0,835	0,378	0,281	0,580	0,069	0,003
Chưa xét nghiệm	DiaNet	0,326	0,837	0,387	0,310	0,513	0,068	0,004
Chỉ HbA1c	LR	0,706	0,935	0,613	0,648	0,581	0,044	0,006
Chỉ HbA1c	HGB	0,763	0,948	0,662	0,761	0,586	0,037	0,004
Chỉ HbA1c	DiaNet	0,769	0,949	0,666	0,735	0,609	0,037	0,003
Chỉ đường huyết	LR	0,616	0,900	0,528	0,567	0,493	0,051	0,009
Chỉ đường huyết	HGB	0,687	0,930	0,578	0,661	0,514	0,044	0,005
Chỉ đường huyết	DiaNet	0,678	0,923	0,575	0,701	0,488	0,045	0,007
Đủ hai xét nghiệm	LR	0,812	0,961	0,722	0,840	0,632	0,034	0,005
Đủ hai xét nghiệm	HGB	0,878	0,977	0,806	0,985	0,682	0,024	0,004
Đủ hai xét nghiệm	DiaNet	0,873	0,975	0,800	0,951	0,690	0,025	0,003

Một DiaNet duy nhất đạt kết quả gần với HistGradientBoosting (HGB), trong khi HGB phải huấn luyện riêng cho từng trường hợp. Khi đủ hai xét nghiệm, DiaNet có PR-AUC 0,873, HGB có 0,878. DiaNet không vượt HGB ở mọi trường hợp; lợi ích của nó là một mô hình dùng được cho cả bốn

trạng thái mà RL đi qua. Ba seed cho độ lệch chuẩn PR-AUC không quá 0,002. ECE dưới 0,01 nên không cần hiệu chỉnh thêm.

Bảng cũng cho thấy giá trị của xét nghiệm: không có xét nghiệm, PR-AUC chỉ 0,326. Khoảng cách này tạo ra bài toán quyết định cho RL ở Mục 4.3.



Hình 4.4 Đường Precision–Recall và độ hiệu chỉnh của DiaNet trên tập test.

4.1.3 RiskNet: dự đoán

Trên bảng, Deep Learning cho hai đầu ra là dự đoán và phân loại. RiskNet học đa nhiệm trên bộ 50.000 bản ghi: một head hồi quy điểm Diabetes_Risk_Score từ 0 đến 100, một head phân loại Low, Moderate, High. Mạng dùng 24 đặc trưng, mỗi đặc trưng có mask riêng, nên có 48 đầu vào và ba lớp ẩn 256, 128, 64. Giá trị thiếu sẵn có trong dữ liệu được che, và 15% đặc trưng bị che thêm ngẫu nhiên khi huấn luyện. Cột AI_Health_Recommendation và Doctor_Consultation_Needed bị loại để tránh rò rỉ nhãn.

In [9]:
```
risk = pd.read_csv(DATA / "diabetes_risk_50k.csv")
ORD = {"Physical_Activity_Level": ["Low", "Moderate", "High"], "Diet_Quality": ["Poor", "Average", "Healthy"],
       "Sugar_Intake_Level": ["Low", "Moderate", "High"], "Stress_Level": ["Low", "Moderate", "High"],
       "Smoking_Status": ["Never", "Former", "Current"], "Alcohol_Consumption": ["Never", "Occasionally", "Frequently"]}
BIN = ["Family_History_Diabetes", "Hypertension", "Heart_Disease", "Fatty_Liver", "PCOS"]
NUM = ["Age", "BMI", "Waist_Circumference_cm", "HbA1c", "Fasting_Blood_Sugar", "Blood_Pressure_Systolic",
       "Exercise_Hours_Per_Week", "Sleep_Hours", "Insulin_Level", "Triglycerides", "HDL", "LDL", "Total_Cholesterol"]
RISK_COLS = NUM + list(ORD) + BIN
TIERS = ["Low", "Moderate", "High"]

def risk_frame(d):
    X = d[NUM].astype(float).copy()
    for c, levels in ORD.items():
        X[c] = d[c].map({v: i for i, v in enumerate(levels)}).astype(float)
    for c in BIN:
        X[c] = d[c].map({"No": 0., "Yes": 1.})
    return X[RISK_COLS]

R = risk_frame(risk)
yr, yc = risk.Diabetes_Risk_Score.values.astype(np.float32), risk.Diabetes_Risk.map({t: i for i, t in enumerate(TIERS)}).values
idx_tr, idx_tmp = train_test_split(np.arange(len(R)), test_size=.30, random_state=SEED, stratify=yc)
idx_va, idx_te = train_test_split(idx_tmp, test_size=.50, random_state=SEED, stratify=yc[idx_tmp])
R_mu, R_sd = R.iloc[idx_tr].mean(), R.iloc[idx_tr].std()

def risk_input(frame, keep=None):
    """Chuẩn hóa, điền 0 cho giá trị thiếu và nối mask đã biết/chưa biết."""
    Xz = ((frame - R_mu) / R_sd).astype(np.float32)
    M = Xz.notna().astype(np.float32).values
    if keep is not None:
        M = M * keep
    return np.hstack([np.nan_to_num(Xz.values) * M, M]).astype(np.float32)

def train_risknet(seed=SEED, epochs=80, batch=256, p_drop=.15, patience=10):
    set_seed(seed)
    Xz = ((R.iloc[idx_tr] - R_mu) / R_sd).astype(np.float32)
    V, M0 = torch.tensor(np.nan_to_num(Xz.values)), torch.tensor(Xz.notna().astype(np.float32).values)
    ys, yt = torch.tensor(yr[idx_tr] / 100), torch.tensor(yc[idx_tr])
    Xv, wv = torch.tensor(risk_input(R.iloc[idx_va])), yr[idx_va]
    model = MaskedMLP(V.shape[1] * 2, {"score": 1, "tier": 3}, hidden=(256, 128, 64), dropout=0.)
    w = torch.tensor(len(yt) / (3 * np.bincount(yc[idx_tr])), dtype=torch.float32)
    ce, hub = nn.CrossEntropyLoss(weight=w), nn.SmoothL1Loss(beta=.05)
    opt = torch.optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-5)
    best, state, wait = 1e9, None, 0
    for epoch in range(epochs):
        model.train(); perm = torch.randperm(len(ys))
        for i in range(0, len(ys), batch):
            b = perm[i:i + batch]
            M = M0[b] * (torch.rand_like(M0[b]) > p_drop).float()  # che ngẫu nhiên đặc trưng
            out = model(torch.cat([V[b] * M, M], 1))
            loss = 10 * hub(out["score"].squeeze(1), ys[b]) + .2 * ce(out["tier"], yt[b])
            opt.zero_grad(); loss.backward(); opt.step()
        model.eval()
        with torch.no_grad():
            mae = np.abs(model(Xv)["score"].squeeze(1).numpy() * 100 - wv).mean()
        if mae < best - 1e-3:
            best, wait, state = mae, 0, {k: v.clone() for k, v in model.state_dict().items()}
        else:
            wait += 1
            if wait >= patience: break
    model.load_state_dict(state)
    return model, epoch + 1

t0 = time.time()
risknet, ep = train_risknet()
torch.save(risknet.state_dict(), RES / "risknet.pt")
print(f"RiskNet dừng ở epoch {ep}, {time.time() - t0:.0f} giây")
```

```
RiskNet dừng ở epoch 77, 48 giây
```

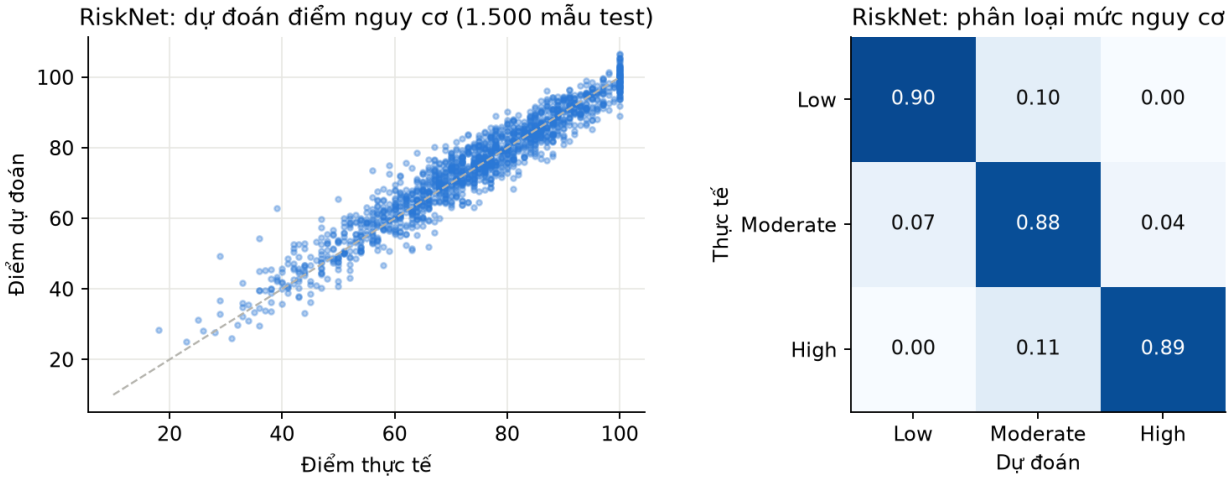
Hình 4.5 Cell [9] chuẩn bị dữ liệu và huấn luyện đa nhiệm RiskNet.

Bảng 4.2 Kết quả RiskNet trên tập test (7.500 dòng)

Mô hình	Đầu vào	MAE	RMSE	R^2	Accuracy	Macro-F1
RiskNet	Đủ đặc trưng	3,42	4,39	0,922	0,886	0,730
RiskNet	Không có xét nghiệm	9,26	11,60	0,455	0,683	0,470
HGB	Đủ đặc trưng	2,67	3,43	0,952	0,901	0,765
HGB	Không có xét nghiệm	9,38	11,21	0,491	0,701	0,475

Với đủ đặc trưng, RiskNet đạt MAE 3,42 và R^2 0,922, kém HGB (MAE 2,67). Lý do chấp nhận RiskNet là mạng xử lý được hồ sơ thiếu theo cùng một cách với DiaNet và thuộc yêu cầu Deep

Learning, không phải vì chính xác hơn. Khi không có xét nghiệm, cả hai giảm mạnh (MAE khoảng 9,3), nên điểm này chỉ dùng tham khảo.



Hình 4.6 RiskNet: điểm dự đoán so với thực tế và ma trận nhầm lẫn mức nguy cơ.

Bảng 4.3 Đổi điểm trung bình khi chỉ thay một đặc trưng (3.000 mẫu test)

Thay đổi	Đổi điểm
Vận động Low → High	-6,26
Ăn uống Poor → Healthy	-4,91
Hút thuốc Current → Never	-4,76
BMI 32 → 24	-4,90

Chiều thay đổi đúng với kỳ vọng. Khi tư vấn, hệ thống chỉ mô phỏng yếu tố người dùng đã khai. Thử nghiệm ban đầu cho thấy mô phỏng yếu tố chưa khai tạo ra số nhiễu có dấu ngược, nên phần này đã bị bỏ. Điểm rủi ro của bộ dữ liệu là một điểm tổng hợp, không phải kết cục lâm sàng, và câu trả lời luôn ghi đây là tương quan, không phải hiệu quả điều trị.

4.1.4 Đưa mô hình vào agent

Prediction Agent nhận hồ sơ, chọn trường hợp theo xét nghiệm đã biết, trả xác suất, nhãn, điểm nguy cơ và các yếu tố ảnh hưởng. Yếu tố ảnh hưởng tính bằng cách thay một giá trị về mốc tham chiếu rồi đo độ đổi xác suất. Đây là phân tích độ nhạy đơn giản của mô hình, không phải quan hệ nhân quả.

4.2 KB + RAG

4.2.1 Tạo KB từ web và sách

KB có 78 ghi chú tiếng Việt. Trong đó 37 ghi chú về loại bệnh, thuốc và triệu chứng kế thừa từ Tiểu luận 03 [18][22]; 41 ghi chú được viết mới từ các trang NIDDK, WHO [10]–[16] và từ sách Physical Activity Guidelines for Americans, 2nd edition [17] với số trang PDF. Mỗi ghi chú là phần diễn giải ngắn, không chép nguyên văn, và lưu nhà xuất bản, URL, ngày truy cập, vị trí trong sách.

Một số nguồn không truy cập được bằng script (trang có kiểm tra reCAPTCHA hoặc trả lỗi 403) nên đã bỏ qua, không tìm cách vượt qua.

Bảng 4.4 Thống kê KB

Hạng mục	Số lượng
Ghi chú (passage)	78
Ghi chú từ web	72
Ghi chú từ sách	6
URL nguồn khác nhau	20
Thực thể	156
Quan hệ	183
Loại quan hệ	58
Loại thực thể	10
Ngưỡng có cấu trúc	11

Đồ thị đáp ứng yêu cầu tối thiểu của bài giảng là 50 thực thể, 100 quan hệ, 5 loại quan hệ, 3 loại thực thể. Các cột dữ liệu được nối với thực thể tri thức qua quan hệ ánh_xạ_tới, là chỗ nhánh DB gặp nhánh KB.

In [13]:
```
docs = json.loads((DATA / "kb_documents.json").read_text(encoding="utf-8"))
DOC = {d["doc_id"]: d for d in docs}
triples = pd.read_csv(DATA / "kb_triples.csv")
thresholds = pd.read_csv(DATA / "kb_thresholds.csv")

G = nx.MultiDiGraph()
for t in triples.itertuples():
    G.add_node(t.subject, kind=t.subject_type); G.add_node(t.object, kind=t.object_type)
    G.add_edge(t.subject, t.object, relation=t.relation, doc_id=t.doc_id)

kinds = Counter(nx.get_node_attributes(G, "kind").values())
kb_stats = {"Ghi chú (passage)": len(docs), "Ghi chú từ web": sum(d["source_type"] == "web" for d in docs),
            "Ghi chú từ sách": sum(d["source_type"] == "book" for d in docs), "URL nguồn khác nhau": len({d["url"] for d in docs}),
            "Thực thể": G.number_of_nodes(), "Quan hệ": G.number_of_edges(), "Loại quan hệ": triples.relation.nunique(),
            "Loại thực thể": len(kinds), "Ngưỡng có cấu trúc": len(thresholds)}
assert G.number_of_nodes() >= 50 and G.number_of_edges() >= 100 and triples.relation.nunique() >= 5 and len(kinds) >= 3
pd.Series(kb_stats, name="Số lượng").to_csv(RES / "kb_stats.csv")
display(pd.Series(kb_stats, name="Số lượng").to_frame())
print(dict(kinds.most_common()))
thresholds
```

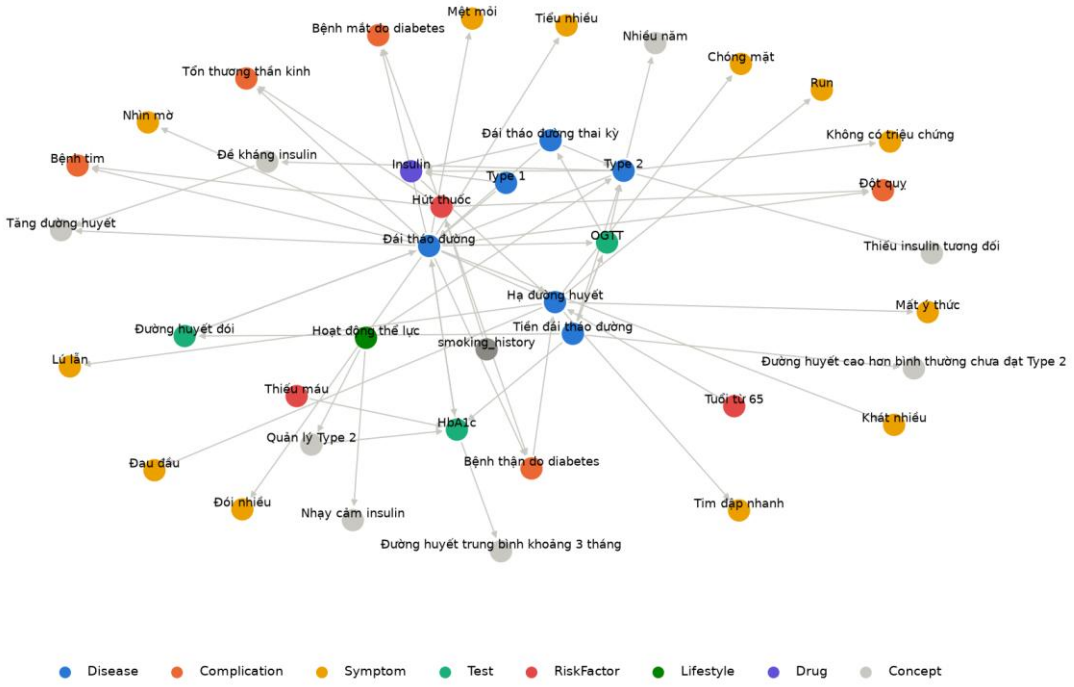
	Số lượng
Ghi chú (passage)	78
Ghi chú từ web	72
Ghi chú từ sách	6
URL nguồn khác nhau	20
Thực thể	156
Quan hệ	183
Loại quan hệ	58
Loại thực thể	10
Ngưỡng có cấu trúc	11

```
{'Concept': 49, 'Symptom': 16, 'Complication': 15, 'RiskFactor': 15, 'Test': 13, 'Lifestyle': 12, 'Disease': 11, 'Drug': 10, 'DrugClass':
8, 'Feature': 7}
```

Out[13]:

	test	category	lower	upper	unit	doc_id
0	HbA1c	Bình thường	NaN	5.7	%	N01
1	HbA1c	Tiền đái tháo đường	5.7	6.5	%	N01
2	HbA1c	Ngưỡng đái tháo đường	6.5	NaN	%	N01
3	Đường huyết đói	Bình thường	NaN	100.0	mg/dL	N02
4	Đường huyết đói	Tiền đái tháo đường	100.0	126.0	mg/dL	N02
5	Đường huyết đói	Ngưỡng đái tháo đường	126.0	NaN	mg/dL	N02
6	OGTT 2 giờ	Bình thường	NaN	140.0	mg/dL	N03
7	OGTT 2 giờ	Tiền đái tháo đường	140.0	200.0	mg/dL	N03
8	OGTT 2 giờ	Ngưỡng đái tháo đường	200.0	NaN	mg/dL	N03
9	Đường huyết ngẫu nhiên	Ngưỡng đái tháo đường (khi có triệu chứng)	200.0	NaN	mg/dL	N04
10	Đường huyết	Hạ đường huyết	NaN	70.0	mg/dL	N15

Hình 4.7 Cell [13] nạp KB, dựng đồ thị và thống kê.



Hình 4.8 Một phần Knowledge Graph quanh các thực thể chính.

Bảng 4.5 Ngưỡng có cấu trúc dùng cho Rule [10][12]

Xét nghiệm	Bình thường	Tiền đái tháo đường	Ngưỡng đái tháo đường
HbA1c	dưới 5,7%	5,7–6,4%	từ 6,5%
Đường huyết đói	dưới 100 mg/dL	100–125 mg/dL	từ 126 mg/dL
OGTT 2 giờ	dưới 140 mg/dL	140–199 mg/dL	từ 200 mg/dL
Đường huyết ngẫu nhiên	không áp dụng	không áp dụng	từ 200 mg/dL khi có triệu chứng

Đồ thị được truy vấn bằng NetworkX. Notebook có 5 truy vấn kèm câu Cypher tương đương và xuất tệp kb_import.cypher. Các câu Cypher này chưa được chạy trên Neo4j.

```
In [14]: def out_of(node, relation=None):
             return [(node, e["relation"], o, e["doc_id"]) for _, o, e in G.out_edges(node, data=True) if relation in (None, e["relation"])]

         def into(node, relation=None):
             return [(s, e["relation"], node, e["doc_id"]) for s, _, e in G.in_edges(node, data=True) if relation in (None, e["relation"])]

         cypher_queries = [
          ("Triệu chứng của hạ đường huyết", "MATCH (:Entity {name:'Hạ đường huyết'})-[:có_thể_có_triệu_chứng]->(s) RETURN s.name",
           [o for _, _, o, _ in out_of("Hạ đường huyết", "có_thể_có_triệu_chứng")]),
          ("Xét nghiệm chẩn đoán đái tháo đường", "MATCH (:Entity {name:'Đái tháo đường'})-[:chẩn_đoán_bằng]->(t) RETURN t.name",
           [o for _, _, o, _ in out_of("Đái tháo đường", "chẩn_đoán_bằng")]),
          ("Lối sống giúp phòng type 2", "MATCH (l)-[:giúp_phòng_hoặc_trì_hoãn]->(:Entity {name:'Type 2'}) RETURN l.name",
           [s for s, _, _, _ in into("Type 2", "giúp_phòng_hoặc_trì_hoãn")]),
          ("Biến chứng hút thuốc làm tăng nguy cơ", "MATCH (:Entity {name:'Hút thuốc'})-[:tăng_nguy_cơ]->(c) RETURN c.name",
           [o for _, _, o, _ in out_of("Hút thuốc", "tăng_nguy_cơ")]),
          ("Cột dữ liệu nối với thực thể tri thức", "MATCH (f:Feature)-[:ánh_xạ_tới]->(e) RETURN f.name, e.name",
           [f"{s} → {o}" for s, _, o in triples.loc[triples.relation == "ánh_xạ_tới", ["subject", "relation", "object"]].values]),
         ]
         for title, cy, ans in cypher_queries:
             print(f"• {title}\n  {cy}\n  → {ans}\n")

         esc = lambda s: str(s).replace("\\", "\\\\").replace("'", "\\'")
         with open(RES / "kb_import.cypher", "w", encoding="utf-8") as fh:
             for n, a in G.nodes(data=True):
                 fh.write(f"MERGE (:Entity:{a['kind']} {{name:'{esc(n)}'}});\n")
             for t in triples.itertuples():
                 fh.write(f"MATCH (a:Entity {{name:'{esc(t.subject)}'}}),(b:Entity {{name:'{esc(t.object)}'}}) MERGE (a)-[:`{t.relation}`
         {{doc_id:'{t.doc_id}'}}]->(b);\n")

• Triệu chứng của hạ đường huyết
  MATCH (:Entity {name:'Hạ đường huyết'})-[:có_thể_có_triệu_chứng]->(s) RETURN s.name
  → ['Run', 'Chóng mặt', 'Lú lẫn', 'Tim đập nhanh', 'Đau đầu', 'Mất ý thức', 'Co giật']

• Xét nghiệm chẩn đoán đái tháo đường
  MATCH (:Entity {name:'Đái tháo đường'})-[:chẩn_đoán_bằng]->(t) RETURN t.name
  → ['HbA1c', 'Đường huyết đói', 'OGTT', 'Đường huyết ngẫu nhiên']

• Lối sống giúp phòng type 2
  MATCH (l)-[:giúp_phòng_hoặc_trì_hoãn]->(:Entity {name:'Type 2'}) RETURN l.name
  → ['Hoạt động thể lực', 'Quản lý cân nặng', 'Giảm 5–7% cân nặng', 'Giảm đồ uống có đường', 'Giảm khẩu phần', 'Giữ cân nặng bình thường',
'Tránh thuốc lá']

• Biến chứng hút thuốc làm tăng nguy cơ
  MATCH (:Entity {name:'Hút thuốc'})-[:tăng_nguy_cơ]->(c) RETURN c.name
  → ['Bệnh tim', 'Đột quỵ', 'Bệnh thận do diabetes', 'Bệnh mắt do diabetes', 'Tổn thương thần kinh']

• Cột dữ liệu nối với thực thể tri thức
  MATCH (f:Feature)-[:ánh_xạ_tới]->(e) RETURN f.name, e.name
  → ['age → Tuổi từ 35', 'bmi → Thừa cân hoặc béo phì', 'HbA1c_level → HbA1c', 'blood_glucose_level → Đường huyết đói', 'blood_glucose_leve
l → Đường huyết ngẫu nhiên', 'smoking_history → Hút thuốc', 'hypertension → Huyết áp cao', 'heart_disease → Bệnh tim']
```

Hình 4.9 Cell [14] năm truy vấn đồ thị và câu Cypher tương đương.

4.2.2 RAG lai

RAG đưa tài liệu truy xuất vào quá trình tạo câu trả lời để giảm nội dung không có căn cứ [4]. Đồ án chạy ba bộ truy xuất trên cùng tập ghi chú và hợp hạng bằng Reciprocal Rank Fusion [6]:

- **Lexical:** TF-IDF từ và cụm hai từ sau khi bỏ dấu và chuẩn hóa từ đồng nghĩa như tiểu đường, đái tháo đường, tuýp, type.
- **Vector:** TF-IDF ký tự giảm chiều bằng LSA [7] còn 64 chiều, so khớp cosine, lưu trong kb_vectors.npz.
- **KG:** liên kết thực thể trong câu hỏi, chấm ghi chú theo quan hệ của thực thể đó, theo tinh thần GraphRAG [5].

Knowledge Agent tự mở rộng truy vấn bằng các thực thể lân cận trong KG khi điểm khớp thấp, và đánh dấu nguồn đã quá 730 ngày. Nếu vẫn không có bằng chứng, Advisor nói chưa đủ thông tin.

In [16]:
```
SYN = [("tieu duong", "dai thao duong"), ("tuyp", "type"), ("typ ", "type "), ("tip ", "type "), ("a1c", "hba1c"), ("hbhba1c", "hba1c"),
       ("tut duong", "ha duong huyet"), ("tut duong huyet", "ha duong huyet"), ("tap the duc", "van dong hoat dong the luc"),
       ("nuoc ngot", "do uong co duong"), ("thuoc la", "hut thuoc"), ("mang bau", "mang thai thai ky"), ("bau", "thai ky"),
       ("ngat", "mat y thuc"), ("glucose", "duong huyet glucose"), ("xu ly", "xu tri")]

def norm(s):
    s = str(s).lower().replace("đ", "d")
    return "".join(c for c in unicodedata.normalize("NFD", s) if unicodedata.category(c) != "Mn")

def expand(q):
    s = norm(q)
    for a, b in SYN:
        s = s.replace(a, b)
    return s

ENTITY_KEYS = {norm(n).replace("diabetes", "dai thao duong"): n for n in G.nodes if len(n) >= 3 and G.nodes[n]["kind"] != "Feature"}

def link_entities(q):
    s = expand(q)
    found = [n for k, n in ENTITY_KEYS.items() if re.search(rf"(?<!\w){re.escape(k)}(?!\w)", s)]
    return sorted(set(found), key=len, reverse=True)

class HybridRetriever:
    def __init__(self, docs, dims=64):
        self.docs = docs
        self.ids = [d["doc_id"] for d in docs]
        text = [expand(d["title"] + ". " + d["text"]) for d in docs]
        self.word = TfidfVectorizer(ngram_range=(1, 2), sublinear_tf=True)
        self.W = self.word.fit_transform(text)
        self.char = TfidfVectorizer(analyzer="char_wb", ngram_range=(3, 5), sublinear_tf=True)
        C = self.char.fit_transform(text)
        self.svd = TruncatedSVD(dims, random_state=SEED).fit(C)
        self.V = self._unit(self.svd.transform(C))
        self.edge_docs = defaultdict(set)
        for t in triples.itertuples():
            self.edge_docs[t.subject].add(t.doc_id); self.edge_docs[t.object].add(t.doc_id)

    @staticmethod
    def _unit(X):
        return X / np.clip(np.linalg.norm(X, axis=1, keepdims=True), 1e-9, None)

    def lexical(self, q):
        return (self.W @ self.word.transform([expand(q)]).T).toarray().ravel()

    def vector(self, q):
        return self.V @ self._unit(self.svd.transform(self.char.transform([expand(q)])))[0]

    def kg(self, q):
        ents = link_entities(q)
        score = np.zeros(len(self.ids))
        for e in ents:
            for i, d in enumerate(self.ids):
                score[i] += d in self.edge_docs[e]
        return score / max(1, len(ents))

    def search(self, q, k=5, mode="hybrid", rrf_k=30):
        scores = {"lexical": self.lexical(q), "vector": self.vector(q), "kg": self.kg(q)}
        if mode != "hybrid":
            rank = np.argsort(-scores[mode], kind="stable")
            return [(self.ids[i], float(scores[mode][i])) for i in rank[:k] if scores[mode][i] > 0]
        fused = np.zeros(len(self.ids))
        for name, weight in (("lexical", 1.0), ("vector", .7), ("kg", .5)):
            order = np.argsort(-scores[name], kind="stable")
            for r, i in enumerate(order):
                if scores[name][i] > 0:
                    fused[i] += weight / (rrf_k + r + 1)
        rank = np.argsort(-fused, kind="stable")
        return [(self.ids[i], float(scores["lexical"][i])) for i in rank[:k] if fused[i] > 0]
```

Hình 4.10 Cell [16] chuẩn hóa tiếng Việt và bộ truy xuất lai (phần 1/2).

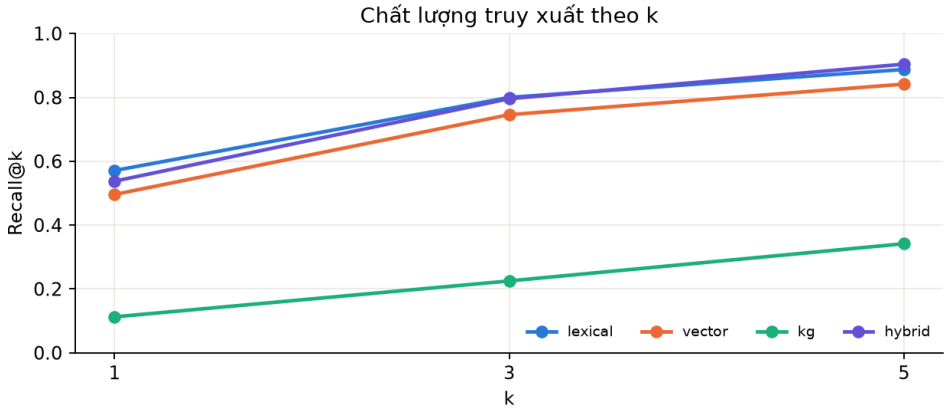
```
retriever = HybridRetriever(docs)
np.savez_compressed(RES / "kb_vectors.npz", vectors=retriever.V.astype(np.float32), ids=np.array(retriever.ids))
print("Vector store:", retriever.V.shape, "| Thực thể trong câu mẫu:", link_entities("Tiểu đường tuýp 2 có cần xét nghiệm HbA1c không?"))
for doc_id, score in retriever.search("tôi bị tụt đường huyết thì xử lý thế nào", k=3):
    print(doc_id, round(score, 3), DOC[doc_id]["title"])
```

```
Vector store: (78, 64) | Thực thể trong câu mẫu: ['Đái tháo đường', 'Type 2', 'HbA1c']
N17 0.274 Ai dễ bị hạ đường huyết
N19 0.217 Hạ đường huyết nặng
N15 0.156 Hạ đường huyết là gì
```

Hình 4.11 Cell [16] chuẩn hóa tiếng Việt và bộ truy xuất lai (phần 2/2).

Bảng 4.6 Chất lượng truy xuất trên 40 câu hỏi có gold

Chế độ	Recall@1	Recall@3	Recall@5	MRR@5	Hit@5
lexical	0,571	0,800	0,887	0,794	0,925
vector	0,496	0,746	0,842	0,733	0,875
kg	0,113	0,225	0,342	0,223	0,425
hybrid	0,537	0,796	0,904	0,765	0,925



Hình 4.12 Recall@k theo chế độ truy xuất.

Kết quả không cho thấy truy xuất lai vượt trội rõ rệt. Hybrid có Recall@5 cao nhất (0,904) nhưng thấp hơn lexical ở Recall@1 (0,537 so với 0,571) và MRR. KG dùng riêng rất yếu vì chỉ nhận ra thực thể có tên trong câu hỏi. Giá trị của KG trong hệ thống nằm ở quan hệ tường minh và bảng ngưỡng để đối chiếu bằng code, hơn là ở điểm xếp hạng. Với kho nhỏ và câu hỏi có từ khóa, lexical đã là mốc mạnh.

4.3 Reinforcement Learning + Rule

4.3.1 Mô hình hóa bài toán

Sau khi có xác suất, agent phải chọn: yêu cầu thêm xét nghiệm hay dừng để đưa khuyến nghị. Bỏ sót nguy hiểm nhưng xét nghiệm cho mọi người thì tốn kém. Đây là bài toán quyết định tuần tự.

Bảng 4.7 Mô hình MDP của Policy Agent

Thành phần	Định nghĩa
Trạng thái	Xét nghiệm đã biết (4 trường hợp) × nhóm xác suất của DiaNet (6 nhóm) = 24 trạng thái
Hành động	ORDER_HBA1C, ORDER_GLUCOSE, ADVISE_ROUTINE, REFER_DOCTOR
Chuyển trạng thái	Xét nghiệm mở ra giá trị thật của bệnh nhân trong dữ liệu, DiaNet tính lại xác suất
Phần thưởng	HbA1c −1; đường huyết −0,5; chuyển khám đúng +20; chuyển khám thừa −4; tư vấn thường đúng +2; bỏ sót −50
Huấn luyện	150.000 episode, 5 seed, $\gamma = 1$, ε giảm từ 1 xuống 0,05, α giảm theo số lần thăm

Các mức phần thưởng là giả định thiết kế của đồ án, không phải chi phí y tế thực. Q-learning học trên tập validation vì DiaNet không học trọng số trên tập này, và được đánh giá trên tập test. Mốc so sánh gồm các chính sách cố định, chính sách chỉ dùng Rule và nghiệm quy hoạch động trên MDP ước lượng từ validation.

In [19]:
```
ACTIONS = ["ORDER_HBA1C", "ORDER_GLUCOSE", "ADVISE_ROUTINE", "REFER_DOCTOR"]
A_H, A_G, A_ROUTINE, A_REFER = range(4)
EDGES = [.02, .05, .10, .30, .60]
NB = len(EDGES) + 1
N_STATES = 4 * NB
REWARD = dict(cost_h=1., cost_g=.5, tp=20., fn=-50., tn=2., fp=-4.)

def state_bins(P):
    return np.column_stack([np.digitize(P[s], EDGES) for s in SCEN])  # cột = known 0..3

def legal(known):
    return [a for a in range(4) if not (a == A_H and known & 1) and not (a == A_G and known & 2)]

def terminal_reward(a, y, R):
    if a == A_REFER:
        return R["tp"] if y else R["fp"]
    return R["fn"] if y else R["tn"]

def train_q(B, y, seed, episodes=150_000, gamma=1., R=REWARD, eps_end=.05, log_every=5000):
    rng = np.random.default_rng(seed)
    Q, N = np.zeros((N_STATES, 4)), np.zeros((N_STATES, 4))
    for s in range(N_STATES):
        for a in range(4):
            if a not in legal(s // NB):
                Q[s, a] = -1e9
    Bl, yl = B.tolist(), y.tolist()
    patients = rng.integers(0, len(yl), episodes).tolist()
    explore = rng.random((episodes, 3)).tolist()
    picks = rng.integers(0, 12, (episodes, 3)).tolist()
    curve, acc = [], 0.
    for ep, i in enumerate(patients):
        eps = max(eps_end, 1 - ep / (.6 * episodes))
        known, total = 0, 0.
        for step in range(3):
            s = known * NB + Bl[i][known]
            acts = legal(known)
            a = acts[picks[ep][step] % len(acts)] if explore[ep][step] < eps else int(np.argmax(Q[s]))
            if a in (A_H, A_G):
                r = -R["cost_h"] if a == A_H else -R["cost_g"]
                known |= 1 if a == A_H else 2
                s2 = known * NB + Bl[i][known]
                target = r + gamma * Q[s2].max()
            else:
                r = terminal_reward(a, yl[i], R)
                target = r
            N[s, a] += 1
            Q[s, a] += max(.002, 1 / (1 + N[s, a]) ** .7) * (target - Q[s, a])
            total += r
            if a >= A_ROUTINE:
                break
        acc += total
        if (ep + 1) % log_every == 0:
            curve.append((ep + 1, acc / log_every)); acc = 0.
    return Q, N, np.array(curve)

B_val, B_test = state_bins(P_val), state_bins(P_test)
t0 = time.time()
RL_SEEDS = [11, 23, 42, 57, 89]
q_runs = {s: train_q(B_val, y_val, s) for s in RL_SEEDS}
Q, Q_N, _ = q_runs[42]
PI = Q.argmax(1)
np.save(RES / "q_table.npy", Q)
print(f"5 seed × 150.000 episode trong {time.time() - t0:.0f} giây")
pd.DataFrame(np.where(Q < -1e8, np.nan, Q).round(2), columns=ACTIONS,
             index=[f"{SCEN[s // NB]}|p{s % NB}" for s in range(N_STATES)]).assign(policy=[ACTIONS[a] for a in PI]).head(8)
```

```
5 seed × 150.000 episode trong 9 giây
```

Out[19]:

	ORDER_HBA1C	ORDER_GLUCOSE	ADVISE_ROUTINE	REFER_DOCTOR	policy

Hình 4.13 Cell [19] môi trường và thuật toán Q-learning (phần 1/2).

none|p0	1.03	1.17	1.08	-3.85	ORDER_GLUCOSE
none|p1	0.11	0.39	0.27	-3.08	ORDER_GLUCOSE
none|p2	0.47	-0.13	-3.18	-2.56	ORDER_HBA1C
none|p3	1.22	2.11	-6.91	-0.15	ORDER_GLUCOSE
none|p4	5.94	4.88	-19.15	4.80	ORDER_HBA1C
none|p5	18.77	0.00	1.74	-3.17	ORDER_HBA1C
hba1c|p0	NaN	1.32	2.00	-3.92	ADVISE_ROUTINE
hba1c|p1	NaN	0.50	-2.58	-2.54	ORDER_GLUCOSE

Hình 4.14 Cell [19] môi trường và thuật toán Q-learning (phần 2/2).

4.3.2 Rule shield

Bảng 4.8 Các luật ghi đè của Policy Agent

Luật	Điều kiện	Hành động	Căn cứ
R1	HbA1c từ 6,5% hoặc đường huyết đạt ngưỡng (lúc đói từ 126, không rõ từ 200 mg/dL)	Bắt buộc REFER_DOCTOR	[10]
R2	Đường huyết 126–199 mg/dL chưa rõ lúc đói, chưa có HbA1c, RL chọn tư vấn thường	Chuyển thành ORDER_HBA1C	[10]
R3	Từ 35 tuổi, chưa có xét nghiệm nào, RL chọn tư vấn thường	Chuyển thành ORDER_HBA1C	[10]

In [20]:

```
H_DIAG = float(thresholds.query("test=='HbA1c' and category=='Ngưỡng đái tháo đường'").lower.iloc[0])
G_FAST = float(thresholds.query("test=='Đường huyết đói' and category=='Ngưỡng đái tháo đường'").lower.iloc[0])
G_RAND = float(thresholds.query("test=='Đường huyết ngẫu nhiên'").lower.iloc[0])
AGE_SCREEN = 35

def lab_flags(known, h, g, fasting=False):
    hk, gk = (known & 1) > 0, (known & 2) > 0
    g_cut = G_FAST if fasting else G_RAND
    diag = (hk & (h >= H_DIAG)) | (gk & (g >= g_cut))
    unclear = gk & ~hk & (g >= G_FAST) & (g < G_RAND) & (not fasting)
    return hk, gk, diag, unclear

def apply_shield(a, known, h, g, age, fasting=False):
    """Trả về hành động sau luật và mã luật đã kích hoạt cho từng bệnh nhân."""
    a, fired = a.copy(), np.full(len(a), "", dtype=object)
    hk, gk, diag, unclear = lab_flags(known, h, g, fasting)
    r2 = unclear & (a == A_ROUTINE); a[r2] = A_H; fired[r2] = "R2"
    r3 = ~hk & ~gk & (age >= AGE_SCREEN) & (a == A_ROUTINE); a[r3] = A_H; fired[r3] = "R3"
    r1 = diag & (a != A_REFER); a[r1] = A_REFER; fired[r1] = "R1"
    return a, fired

def rule_action(known, h, g, age, bmi, fasting=False):
    """Chính sách chỉ dùng luật, làm mốc so sánh với RL."""
    hk, gk, diag, _ = lab_flags(known, h, g, fasting)
    a = np.full(len(known), A_ROUTINE)
    a[~hk & ((age >= AGE_SCREEN) | (bmi >= 25))] = A_H
    a[hk & ~gk & (h >= 5.7)] = A_G
    a[diag] = A_REFER
    return a

te_h, te_g = test_df.HbA1c_level.values, test_df.blood_glucose_level.values
te_age, te_bmi = test_df.age.values, test_df.bmi.values

def rollout(chooser, B, y, h, g, age, R=REWARD):
    n = len(y); known = np.zeros(n, int); final = np.full(n, -1); cost = np.zeros(n); tests = np.zeros(n)
    active = np.arange(n)
    for _ in range(3):
        if not len(active): break
        k = known[active]
        a = chooser(k, B[active, k], active)
        for act, bit, c in ((A_H, 1, R["cost_h"]), (A_G, 2, R["cost_g"])):
            m = a == act
            known[active[m]] |= bit; cost[active[m]] += c; tests[active[m]] += 1
        done = a >= A_ROUTINE
        final[active[done]] = a[done]
        active = active[~done]
    final[active] = A_REFER
    refer = final == A_REFER
    reward = np.where(refer, np.where(y == 1, R["tp"], R["fp"]), np.where(y == 1, R["fn"], R["tn"])) - cost
    _, _, diag, _ = lab_flags(known, h, g)
    viol = (~refer) & (diag | ((known == 0) & (age >= AGE_SCREEN)))
    tp = (refer & (y == 1)).sum()
    return {"Reward TB": reward.mean(), "Sensitivity": tp / (y == 1).sum(), "Specificity": ((~refer) & (y == 0)).sum() / (y == 0).sum(),
            "Xét nghiệm/người": tests.mean(), "Bỏ sót": int(((~refer) & (y == 1)).sum()), "Vi phạm luật": int(viol.sum())}

def table_chooser(pi, shield=False):
    def choose(k, b, idx):
        a = pi[k * NB + b]
        if shield:
            a, _ = apply_shield(a, k, te_h[idx], te_g[idx], te_age[idx])
        return a
    return choose

def value_iteration(B, y, R=REWARD):
    """Nghiệm tối ưu của MDP thực nghiệm ước lượng từ tập validation, dùng kiểm tra Q-learning."""
    V, pi = np.zeros(N_STATES), np.zeros(N_STATES, int)
    for known in (3, 2, 1, 0):
        for b in range(NB):
            m = B[:, known] == b
```

Hình 4.15 Cell [20] Rule shield, mô phỏng chính sách và bảng so sánh (phần 1/2).

```
            s = known * NB + b
            if not m.any():
                pi[s] = A_ROUTINE; continue
            vals = {A_ROUTINE: np.mean(np.where(y[m] == 1, R["fn"], R["tn"])), A_REFER: np.mean(np.where(y[m] == 1, R["tp"], R["fp"]))}
            for a, bit, c in ((A_H, 1, R["cost_h"]), (A_G, 2, R["cost_g"])):
                if a in legal(known):
                    k2 = known | bit
                    vals[a] = -c + V[k2 * NB + B[m, k2]].mean()
            pi[s] = max(vals, key=vals.get); V[s] = vals[pi[s]]
    return pi, V

PI_DP, V_DP = value_iteration(B_val, y_val)
rng_eval = np.random.default_rng(0)
def random_chooser(k, b, idx):
    return np.array([rng_eval.choice(legal(int(x))) for x in k])

break_even = (REWARD["tn"] - REWARD["fp"]) / (REWARD["tp"] - REWARD["fp"] + REWARD["tn"] - REWARD["fn"])
def fixed_test_chooser(order):
    def choose(k, b, idx):
        a = np.full(len(k), A_ROUTINE)
        for i in range(len(k)):
            nxt = next((t for t in order if not (int(k[i]) & (1 if t == A_H else 2))), None)
            a[i] = nxt if nxt is not None else (A_REFER if P_test[SCEN[int(k[i])]][idx[i]] >= break_even else A_ROUTINE)
        return a
    return choose

rule_chooser = lambda k, b, idx: rule_action(k, te_h[idx], te_g[idx], te_age[idx], te_bmi[idx])
env_test = dict(B=B_test, y=y_test, h=te_h, g=te_g, age=te_age)
rows = [{"Chính sách": n, **rollout(c, **env_test)} for n, c in [
    ("Ngẫu nhiên", random_chooser), ("Không xét nghiệm", fixed_test_chooser([])), ("Luôn HbA1c", fixed_test_chooser([A_H])),
    ("Luôn đủ hai xét nghiệm", fixed_test_chooser([A_H, A_G])), ("Chỉ dùng Rule", rule_chooser),
    ("Quy hoạch động (mốc tối ưu)", table_chooser(PI_DP))]]
for name, sh in (("Q-learning (RL)", False), ("RL + Rule", True)):
    per = pd.DataFrame([rollout(table_chooser(q[0].argmax(1), sh), **env_test) for q in q_runs.values()])
    mean, sd = per.mean(), per.std(ddof=1)
    rows.append({"Chính sách": name, **mean.to_dict(), "SD reward": sd["Reward TB"]})
rl_results = pd.DataFrame(rows)
rl_results.to_csv(RES / "rl_policy_results.csv", index=False)
print(f"Điểm hòa vốn REFER/ROUTINE: p = {break_even:.3f}")
rl_results.round(3)
```

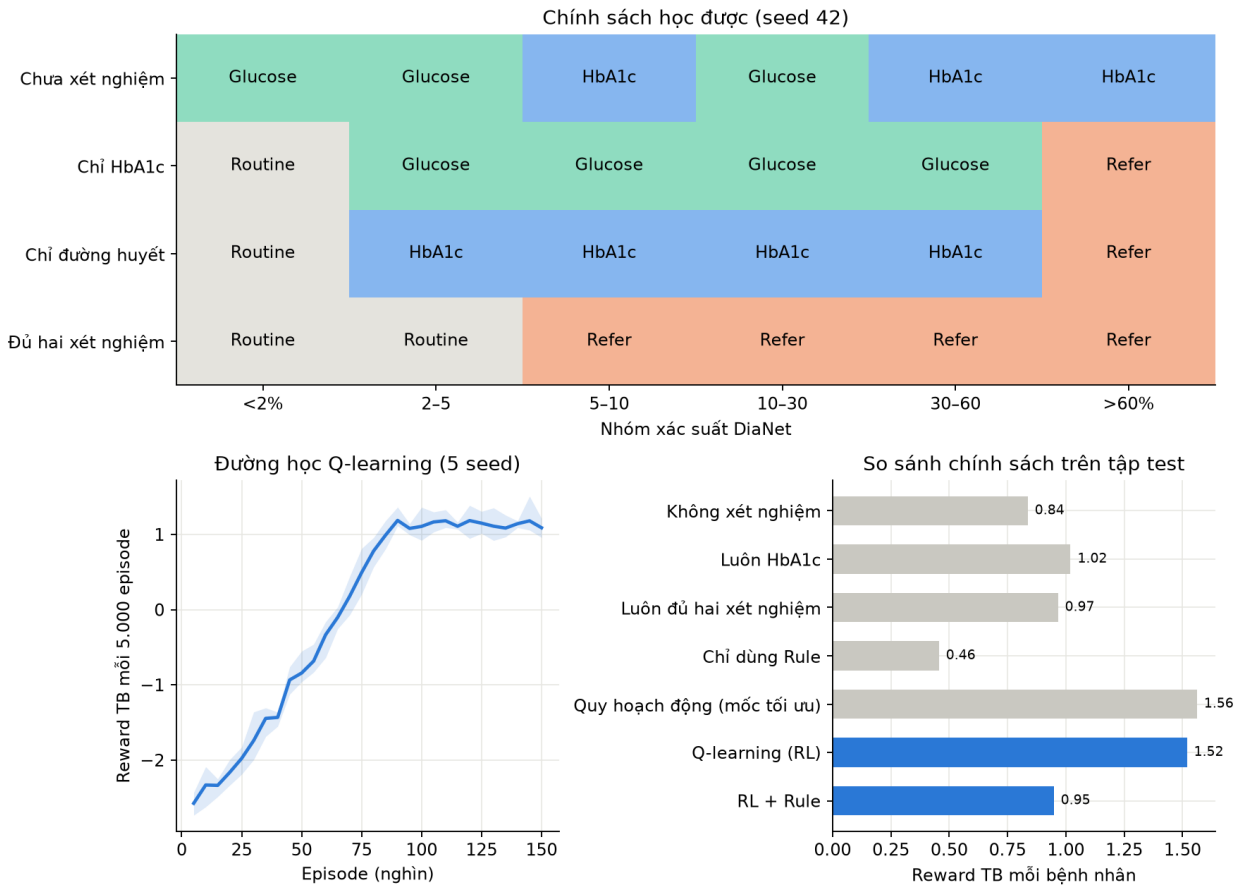
```
Điểm hòa vốn REFER/ROUTINE: p = 0.079
```

Out[20]:

	Chính sách	Reward TB	Sensitivity	Specificity	Xét nghiệm/người	Bỏ sót	Vi phạm luật	SD reward
0	Ngẫu nhiên	-2.717	0.500	0.504	0.665	637.0	2927.0	NaN
1	Không xét nghiệm	0.837	0.826	0.694	0.000	222.0	3842.0	NaN
2	Luôn HbA1c	1.020	0.903	0.822	1.000	123.0	1583.0	NaN
3	Luôn đủ hai xét nghiệm	0.966	0.916	0.890	2.000	107.0	2515.0	NaN
4	Chỉ dùng Rule	0.456	0.789	0.843	1.136	269.0	0.0	NaN
5	Quy hoạch động (mốc tối ưu)	1.562	0.868	0.901	0.941	168.0	1617.0	NaN
6	Q-learning (RL)	1.519	0.934	0.852	1.293	83.8	1102.4	0.069
7	RL + Rule	0.948	0.970	0.740	1.462	37.8	0.0	0.285

Hình 4.16 Cell [20] Rule shield, mô phỏng chính sách và bảng so sánh (phần 2/2).

4.3.3 Kết quả



Hình 4.17 Đường học, chính sách học được và phần thưởng của các chính sách.

Bảng 4.9 So sánh chính sách trên tập test (Q-learning là trung bình 5 seed)

Chính sách	Reward TB	Sensitivity	Specificity	Số xét nghiệm	Bỏ sót	Vi phạm luật
Ngẫu nhiên	-2,72	0,500	0,504	0,66	637,0	2927,0
Không xét nghiệm	0,84	0,826	0,694	0,00	222,0	3842,0
Luôn HbA1c	1,02	0,903	0,822	1,00	123,0	1583,0
Luôn đủ hai xét nghiệm	0,97	0,916	0,890	2,00	107,0	2515,0
Chỉ dùng Rule	0,46	0,789	0,843	1,14	269,0	0,0
Quy hoạch động (mốc tối ưu)	1,56	0,868	0,901	0,94	168,0	1617,0
Q-learning (RL)	1,52	0,934	0,852	1,29	83,8	1102,4
RL + Rule	0,95	0,970	0,740	1,46	37,8	0,0

Q-learning đạt reward 1,52, gần mốc quy hoạch động 1,56, và cao hơn mọi chính sách cố định. Nó dùng 1,29 xét nghiệm mỗi người, ít hơn cách luôn làm đủ hai xét nghiệm, mà sensitivity cao hơn (0,934). Chính sách học được có logic dễ hiểu: nhóm xác suất rất thấp dừng sớm, nhóm trung gian yêu cầu thêm xét nghiệm, nhóm cao chuyển khám (Hình 4.17).

Tuy vậy, Q-learning vẫn để lại trung bình 1102,4 ca vi phạm luật, là các ca tư vấn thường cho người đã đạt ngưỡng hoặc người từ 35 tuổi chưa xét nghiệm. Thêm Rule đưa số này về 0 và giảm bỏ sót từ 83,8 xuống 37,8. Đổi lại, specificity giảm từ 0,852 xuống 0,740 và reward giảm còn 0,95.

Reward giảm không có nghĩa Rule làm hệ thống kém đi. Reward được tính theo nhãn dữ liệu, mà nhãn ở nhóm HbA1c 6,5–6,9% mâu thuẫn với tài liệu y khoa. Hệ thống ưu tiên tài liệu nên chấp nhận mất điểm theo nhãn. So với Rule thuần (reward 0,46, 269 ca bỏ sót), RL + Rule tốt hơn trên cả hai chỉ số, nhưng dùng nhiều xét nghiệm hơn và có specificity thấp hơn. Đây là lý do cần cả hai khối như trên bảng.

Bảng 4.10 Độ nhạy theo mức phạt bỏ sót (seed 42, 100.000 episode)

Phạt bỏ sót	Sensitivity	Specificity	Xét nghiệm/người	Bỏ sót
-20	0,833	0,911	0,76	213
-50	0,936	0,854	1,38	81
-100	0,964	0,789	1,38	46

Tăng mức phạt làm chính sách thận trọng hơn. Vì vậy mức phạt là lựa chọn giá trị cần chuyên gia y tế xác định, không phải tham số kỹ thuật thuần túy.

4.4 Các agent tư vấn, trợ giúp và điều phối

4.4.1 Thông điệp và hợp đồng

In [25]:
```
SCHEMA = {
 "PARSE": ["text", "memory"], "PARSED": ["profile", "intent", "errors", "missing", "new_fields"],
 "PRECHECK": ["text", "profile"], "SAFETY": ["level", "reason"],
 "PREDICT": ["profile"], "PREDICTION": ["p", "label", "scenario", "threshold", "score", "tier", "drivers", "what_if"],
 "DECIDE": ["p", "profile"], "DECISION": ["action", "rl_action", "rules", "lab_checks"],
 "RETRIEVE": ["queries"], "EVIDENCE": ["items", "facts", "stale", "expanded", "tools"],
 "COMPOSE": ["state"], "DRAFT": ["text"], "CHECKLIST": ["state"], "PLAN": ["text"],
 "REVIEW": ["state"], "VERDICT": ["passed", "issues"],
}

@dataclass
class Message:
    sender: str
    receiver: str
    kind: str
    payload: dict
    conversation: str = ""
    message_id: str = field(default_factory=lambda: uuid.uuid4().hex[:8])
    timestamp: float = field(default_factory=time.time)

def validate_message(m):
    """Hợp đồng giữa các agent: sai kiểu hoặc thiếu trường thì coi là lỗi."""
    if not isinstance(m, Message):
        raise TypeError("Phản hồi không phải Message")
    if m.kind not in SCHEMA:
        raise ValueError(f"Kiểu tin nhắn lạ: {m.kind}")
    missing = [k for k in SCHEMA[m.kind] if k not in m.payload]
    if missing:
        raise ValueError(f"{m.kind} thiếu trường {missing}")
    return m

class Agent:
    tools = ()
    def __init__(self, name):
        self.name = name
    def reply(self, msg, kind, **payload):
        return Message(self.name, msg.sender, kind, payload, msg.conversation)
    def handle(self, msg):
        raise NotImplementedError

def vn(x, nd=1):
    return f"{x:.{nd}f}".replace(".", ",")

def pct(p):
    """Không hiển thị 0% hoặc 100% để tránh hiểu xác suất mô hình là chắc chắn."""
    return "trên 99,9%" if p > .999 else ("dưới 0,1%" if p < .001 else f"{vn(p * 100)}%")
```

Hình 4.18 Cell [25] schema thông điệp và lớp Agent.

4.4.2 Intake Agent và Safety Agent

Intake trích tuổi, giới tính, BMI, chiều cao và cân nặng, HbA1c, đường huyết bằng regex tiếng Việt, đổi mmol/L sang mg/dL, kiểm tra miền giá trị và xác định ý định. Safety có hai điểm kiểm tra. Trước khi xử lý, nó phát hiện dấu hiệu khẩn cấp, đường huyết dưới 70 mg/dL và yêu cầu kê đơn. Sau khi có bản nháp, nó kiểm tra trích dẫn, liều thuốc, câu khẳng định bệnh, số liệu không có nguồn và việc nêu chỉ số đạt ngưỡng.

In [26]:

```
NUMBER = r"(\d+(?:[.,]\d+)?)"
to_f = lambda s: float(s.replace(",", "."))
LIMITS = {"age": (1, 120), "bmi": (10, 70), "hba1c": (3, 20), "glucose": (20, 800)}

def parse_profile(text):
    s, p = norm(text), {}
    if m := re.search(rf"{NUMBER}\s*tuoi", s): p["age"] = to_f(m[1])
    if re.search(r"\b(nam|ong|bo toi|anh)\b", s): p["gender"] = "Male"
    if re.search(r"\b(nu|ba|me toi|chi)\b", s): p["gender"] = "Female"
    if m := re.search(rf"bmi\D{{0,6}}{NUMBER}", s): p["bmi"] = to_f(m[1])
    hgt = re.search(r"cao\s*(\d)m(\d{1,2})", s) or re.search(r"cao\s*(\d)[.,](\d{1,2})\s*m", s)
    cm = re.search(r"cao\s*(\d{3})\s*cm", s)
    wgt = re.search(rf"nang\s*{NUMBER}\s*(?:kg|can)", s)
    height = int(hgt[1]) + int(hgt[2].ljust(2, "0")) / 100 if hgt else (int(cm[1]) / 100 if cm else None)
    if height and wgt and "bmi" not in p:
        p["bmi"] = round(to_f(wgt[1]) / height ** 2, 1)
    if m := re.search(rf"(?:hba1c|a1c)\D{{0,12}}{NUMBER}", s): p["hba1c"] = to_f(m[1])
    if m := re.search(rf"(?:duong huyet|glucose)(\s+luc doi|\s+doi|\s+ngau nhien|\s+bat ky)?\D{{0,16}}{NUMBER}\s*(mmol)?", s):
        p["glucose"] = round(to_f(m[2]) * 18, 0) if m[3] else to_f(m[2])
        kind = (m[1] or "").strip()
        p["glucose_kind"] = "fasting" if "doi" in kind else ("random" if kind else "unknown")
    if re.search(r"(?<!khong )(tang huyet ap|cao huyet ap)", s): p["hypertension"] = 1
    if re.search(r"khong (bi )?(tang huyet ap|cao huyet ap)", s): p["hypertension"] = 0
    if re.search(r"(?<!khong )(benh tim|suy tim)", s): p["heart_disease"] = 1
    if re.search(r"khong (hut thuoc|co hut thuoc)", s): p["smoking"] = "never"
    elif re.search(r"(bo|cai|da ngung) thuoc", s): p["smoking"] = "former"
    elif re.search(r"(dang )?hut thuoc", s) and "anh huong" not in s and "?" not in text: p["smoking"] = "current"
    elif re.search(r"dang hut thuoc", s): p["smoking"] = "current"
    if re.search(r"(bo|me|cha|anh|chi|em)( toi)? (bi|mac) (tieu duong|dai thao duong)", s): p["family_history"] = 1
    if re.search(r"it van dong|luoi van dong|ngoi nhieu", s): p["activity"] = "Low"
    return p

KW_SUPPORT = ["chuan bi", "checklist", "danh sach", "mang gi", "hoi bac si gi", "tai kham"]
KW_ASSESS = ["nguy co", "danh gia", "kiem tra giup"]
KW_DOMAIN = ["dai thao duong", "tieu duong", "duong huyet", "insulin", "hba1c", "a1c", "van dong", "metformin", "sglt2", "cgm", "tuyp",
"thuoc tieu duong"]

class IntakeAgent(Agent):
    tools = ("parse_profile",)
    def handle(self, msg):
        text, memory = msg.payload["text"], msg.payload["memory"]
        found, errors = parse_profile(text), []
        for key, (lo, hi) in LIMITS.items():
            if key in found and not lo <= found[key] <= hi:
                errors.append(f"{key}={found[key]:g} nằm ngoài khoảng hợp lệ {lo}-{hi}")
                found.pop(key)
        profile = {**memory.get("profile", {}), **found}
        s = norm(text)
        measured = {k for k in found if k in ("age", "bmi", "hba1c", "glucose")}
        domain = bool(link_entities(text)) or any(k in s for k in KW_DOMAIN)
        if any(k in s for k in KW_SUPPORT) and (domain or memory.get("profile")):
            intent = "support"
        elif any(k in s for k in KW_ASSESS) or measured or errors or memory.get("pending") == "assess":
            intent = "assess"
        elif domain:
            intent = "knowledge"
        else:
            intent = "out_of_scope"
        missing = [k for k in ("age", "bmi") if k not in profile] if intent == "assess" else []
        return self.reply(msg, "PARSED", profile=profile, intent=intent, errors=errors, missing=missing, new_fields=sorted(found))

EMERGENCY = ["bat tinh", "hon me", "co giat", "kho tho", "dau nguc", "khong danh thuc", "ngat xiu", "non lien tuc"]
BOUNDARY = [r"\blieu\b", "ke don", "ke cho", "bao nhieu mg", "may vien", "ngung insulin", "ngung thuoc", "bo thuoc", "doi thuoc", "tang
lieu", "giam lieu"]
DOSE = re.compile(r"\d+(?:[.,]\d+)?\s*(?:mg(?!/dL)|mcg|viên|đơn vị|UI\b|IU\b)(?!/)", re.I)
DIAGNOSIS = re.compile(r"(bạn|anh|chị|ông|bà)\s+(đã\s+)?(bị|mắc)\s+(bệnh\s+)?(đái tháo đường|tiểu đường)", re.I)
UNIT_NUMBER = re.compile(rf"{NUMBER}\s*(%|mg/dL|mmol/L)")
DISCLAIMER = "Thông tin chỉ để tham khảo, không thay thế chẩn đoán của bác sĩ."
```

Hình 4.19 Cell [26] Intake Agent và Safety Agent (phần 1/2).

```
def allowed_numbers(state):
    vals = set()
    for e in state.get("evidence", []):
        vals |= {to_f(x) for x in re.findall(NUMBER, e["text"])}
    prof = state.get("profile", {})
    vals |= {float(prof[k]) for k in ("hba1c", "glucose", "bmi", "age") if k in prof}
    if state.get("prediction"):
        pr = state["prediction"]
        vals |= {round(pr["p"] * 100, 1), round(pr["threshold"] * 100, 1), round(pr["score"], 1), 99.9, .1}
        vals |= {round(abs(d) * 100, 1) for _, d in pr["drivers"]}
    for c in (state.get("decision") or {}).get("lab_checks", []):
        vals.add(float(c["value"]))
    return vals

class SafetyAgent(Agent):
    tools = ("red_flags", "critic")
    def handle(self, msg):
        return self.precheck(msg) if msg.kind == "PRECHECK" else self.review(msg)

    def precheck(self, msg):
        s, p = norm(msg.payload["text"]), msg.payload["profile"]
        if hit := [k for k in EMERGENCY if k in s]:
            return self.reply(msg, "SAFETY", level="emergency", reason=hit[0])
        if p.get("glucose", 999) < 70:
            return self.reply(msg, "SAFETY", level="urgent", reason="đường huyết dưới 70 mg/dL")
        if any(re.search(k, s) for k in BOUNDARY):
            return self.reply(msg, "SAFETY", level="boundary", reason="kê đơn hoặc thay đổi thuốc")
        return self.reply(msg, "SAFETY", level="ok", reason="")

    def review(self, msg):
        st = msg.payload["state"]; draft = st["draft"]; ev = st.get("evidence", []); issues = []
        cited = {int(n) for n in re.findall(r"\[(\d+)\]", draft)}
        if bad := sorted(n for n in cited if not 1 <= n <= len(ev)):
            issues.append(("CITATION", f"Trích dẫn {bad} không tồn tại"))
        if ev and not cited and st.get("needs_citation", True):
            issues.append(("CITATION", "Thiếu trích dẫn cho nội dung y khoa"))
        if DOSE.search(draft):
            issues.append(("DOSE", "Câu trả lời không được nêu liều thuốc"))
        if DIAGNOSIS.search(draft):
            issues.append(("DIAGNOSIS", "Không được khẳng định người dùng mắc bệnh"))
        if DISCLAIMER not in draft and st.get("needs_citation", True):
            issues.append(("DISCLAIMER", "Thiếu câu nhắc tham khảo"))
        ok = allowed_numbers(st)
        for raw, unit in UNIT_NUMBER.findall(draft):
            if not any(abs(to_f(raw) - a) < .051 for a in ok):
                issues.append(("NUMBER", f"Số {raw}{unit} không có nguồn"))
        have = {e["doc_id"] for e in ev}
        for c in (st.get("decision") or {}).get("lab_checks", []):
            if c["doc_id"] not in have:
                issues.append(("NEED_EVIDENCE", c["doc_id"]))
            if c["diagnostic"] and "ngưỡng" not in draft:
                issues.append(("REASSURANCE", f"Phải nêu {c['test']} đạt ngưỡng"))
        return self.reply(msg, "VERDICT", passed=not issues, issues=issues)
```

Hình 4.20 Cell [26] Intake Agent và Safety Agent (phần 2/2).

4.4.3 Prediction, Policy và Knowledge Agent

In [27]:

```
def lab_checks(profile):
    """Công cụ threshold_lookup: đối chiếu chỉ số người dùng với bảng ngưỡng trong KB."""
    out = []
    def row(test, value, unit, category, doc_id, diagnostic=False):
        out.append(dict(test=test, value=value, unit=unit, category=category, doc_id=doc_id, diagnostic=diagnostic))
    if "hba1c" in profile:
        v = profile["hba1c"]
        for t in thresholds[thresholds.test == "HbA1c"].itertuples():
            if (pd.isna(t.lower) or v >= t.lower) and (pd.isna(t.upper) or v < t.upper):
                row("HbA1c", v, "%", t.category, t.doc_id, "Ngưỡng đái" in t.category)
    if "glucose" in profile:
        v, kind = profile["glucose"], profile.get("glucose_kind", "unknown")
        if kind == "fasting":
            for t in thresholds[thresholds.test == "Đường huyết đói"].itertuples():
                if (pd.isna(t.lower) or v >= t.lower) and (pd.isna(t.upper) or v < t.upper):
                    row("Đường huyết lúc đói", v, "mg/dL", t.category, t.doc_id, "Ngưỡng đái" in t.category)
        elif v >= G_RAND:
            row("Đường huyết", v, "mg/dL", "Ngưỡng đường huyết ngẫu nhiên, cần xác nhận", "N04", True)
        elif v >= G_FAST:
            row("Đường huyết", v, "mg/dL", "Chưa rõ đo lúc đói hay không", "N02")
    return out

def dia_vector(p):
    smoke = {"current": "current", "former": "former", "never": "never"}.get(p.get("smoking"), "No Info")
    gender = {"Male": 1., "Female": 0.}.get(p.get("gender"), .5)
    base = np.array([[float(z(p["age"], "age")), float(z(p["bmi"], "bmi")), p.get("hypertension", 0), p.get("heart_disease", 0), gender]
                     + [float(smoke == s) for s in SMOKE]], np.float32)
    h, g = np.array([z(p.get("hba1c", 0), "h")], np.float32), np.array([z(p.get("glucose", 0), "g")], np.float32)
    mh, mg = np.array([float("hba1c" in p)], np.float32), np.array([float("glucose" in p)], np.float32)
    return dia_input(base, h, g, mh, mg)

def dia_prob(p):
    dianet.eval()
    with torch.no_grad():
        return float(torch.sigmoid(dianet(torch.tensor(dia_vector(p)))["cls"]))

def risk_row(p):
    row = {c: np.nan for c in RISK_COLS}
    row.update(Age=p["age"], BMI=p["bmi"])
    if "hba1c" in p: row["HbA1c"] = p["hba1c"]
    if p.get("glucose_kind") == "fasting": row["Fasting_Blood_Sugar"] = p["glucose"]
    if "hypertension" in p: row["Hypertension"] = float(p["hypertension"])
    if "heart_disease" in p: row["Heart_Disease"] = float(p["heart_disease"])
    if "family_history" in p: row["Family_History_Diabetes"] = float(p["family_history"])
    if "smoking" in p: row["Smoking_Status"] = {"never": 0., "former": 1., "current": 2.}[p["smoking"]]
    if "activity" in p: row["Physical_Activity_Level"] = float(ORD["Physical_Activity_Level"].index(p["activity"]))
    return pd.DataFrame([row])[RISK_COLS]

class PredictionAgent(Agent):
    tools = ("DiaNet", "RiskNet")
    def handle(self, msg):
        p = msg.payload["profile"]
        scen = SCEN[int("hba1c" in p) + 2 * int("glucose" in p)]
        prob, thr = dia_prob(p), THRESH[scen]
        score, tier = risk_predict(risk_row(p))
        refs = {"Xét nghiệm HbA1c": ("hba1c", None), "Đường huyết": ("glucose", None), "BMI": ("bmi", 23.), "Tuổi": ("age", 30.),
                "Tăng huyết áp": ("hypertension", 0), "Bệnh tim": ("heart_disease", 0), "Hút thuốc": ("smoking", "never")}
        drivers = []
        for name, (key, ref) in refs.items():
            if key in p and p[key] != ref:
                alt = {k: v for k, v in p.items() if k != key} if ref is None else {**p, key: ref}
                drivers.append((name, prob - dia_prob(alt)))
        drivers = sorted([d for d in drivers if abs(d[1]) >= .005], key=lambda d: -abs(d[1]))[:3]
        what_if = []
        # Chỉ mô phỏng yếu tố người dùng đã khai, vì không biết mức hiện tại thì không có mốc so sánh
        changes = {"giảm BMI về 24": {"BMI": min(24., p["bmi"])}, "ngừng hút thuốc": {"Smoking_Status": 0.}, "vận động mức cao":
{"Physical_Activity_Level": 2.}}
        ref_row, ref_score = risk_row(p), float(score[0])
        for name, ch in changes.items():
```

Hình 4.21 Cell [27] công cụ đối chiếu ngưỡng và ba agent chuyên môn (phần 1/2).

```
            if name == "giảm BMI về 24" and p["bmi"] <= 24: continue
            if name == "ngừng hút thuốc" and p.get("smoking") != "current": continue
            if name == "vận động mức cao" and p.get("activity") != "Low": continue
            alt = ref_row.copy()
            for k, v in ch.items(): alt[k] = v
            what_if.append((name, float(risk_predict(alt)[0][0]) - ref_score))
        return self.reply(msg, "PREDICTION", p=prob, label=int(prob >= thr), scenario=scen, threshold=thr, score=float(score[0]),
                          tier=TIERS[int(tier.argmax())], drivers=drivers, what_if=what_if)

class PolicyAgent(Agent):
    tools = ("Q-table", "threshold_lookup")
    def __init__(self, name, mode="rl+rule"):
        super().__init__(name); self.mode = mode
    def handle(self, msg):
        pr, p = msg.payload["p"], msg.payload["profile"]
        known = np.array([int("hba1c" in p) + 2 * int("glucose" in p)])
        h, g, age, bmi = (np.array([float(p.get(k, 0))]) for k in ("hba1c", "glucose", "age", "bmi"))
        fasting = p.get("glucose_kind") == "fasting"
        rl = PI[known * NB + np.digitize([pr], EDGES)]
        rules = ""
        if self.mode == "rule":
            a = rule_action(known, h, g, age, bmi, fasting)
        elif self.mode == "rl":
            a = rl
        else:
            a, fired = apply_shield(rl, known, h, g, age, fasting); rules = fired[0]
        return self.reply(msg, "DECISION", action=ACTIONS[int(a[0])], rl_action=ACTIONS[int(rl[0])], rules=rules,
lab_checks=lab_checks(p))

class KnowledgeAgent(Agent):
    tools = ("search_kb", "kg_neighbors")
    MIN_SCORE, MAX_AGE_DAYS = .04, 730
    def __init__(self, name, today=None):
        super().__init__(name); self.today = pd.Timestamp(today or "2026-11-05")

    def facts_for(self, text, limit=6):
        facts = []
        for e in link_entities(text)[:3]:
            facts += [f"({s}, {r}, {o}) [{d}]" for s, r, o, d in (out_of(e) + into(e))[:3]]
        return list(dict.fromkeys(facts))[:limit]

    def handle(self, msg):
        items, tools, expanded = {}, [], False
        k = msg.payload.get("k", 3)
        for q in msg.payload["queries"]:
            tools.append("search_kb")
            hits = retriever.search(q, k=k)
            if not hits or hits[0][1] < self.MIN_SCORE:  # bằng chứng yếu: tự mở rộng truy vấn bằng KG
                ents = link_entities(q)
                if ents:
                    tools.append("kg_neighbors"); expanded = True
                    neigh = [o for _, _, o, _ in out_of(ents[0])[:4]] + [s for s, _, _, _ in into(ents[0])[:4]]
                    hits = retriever.search(q + " " + " ".join(neigh), k=k)
            for doc_id, score in hits:
                if score >= self.MIN_SCORE / 2 and doc_id not in items:
                    items[doc_id] = score
        for doc_id in msg.payload.get("doc_ids", []):
            items.setdefault(doc_id, 1.0)
        out = [{**DOC[d], "score": s} for d, s in items.items()]
        stale = [d["doc_id"] for d in out if (self.today - pd.Timestamp(d["accessed"])).days > self.MAX_AGE_DAYS]
        return self.reply(msg, "EVIDENCE", items=out, facts=self.facts_for(" ".join(msg.payload["queries"])), stale=stale,
expanded=expanded, tools=tools)
```

Hình 4.22 Cell [27] công cụ đối chiếu ngưỡng và ba agent chuyên môn (phần 2/2).

4.4.4 Advisor, Support và Coordinator

Advisor là khối **tư vấn**. Nó trình bày phân loại, dự đoán, đối chiếu ngưỡng, bước tiếp theo và các ghi chú liên quan kèm trích dẫn. Khi mô hình báo thấp nhưng chỉ số đạt ngưỡng, Advisor nêu rõ mâu thuẫn. Support là khối **trợ giúp**. Nó lập danh sách chuẩn bị đi khám từ hồ sơ trong bộ nhớ và các nguồn đã truy xuất, theo gợi ý hỗ trợ chuẩn bị khám trong bài giảng [20]. Không có LLM nên câu trả lời được ghép từ mẫu câu và nguyên văn ghi chú. Cách này không tự sinh được số liệu ngoài nguồn, đổi lại văn phong cứng hơn.

In [28]:

```
ACTION_VI = {"ORDER_HBA1C": "nên làm xét nghiệm HbA1c", "ORDER_GLUCOSE": "nên đo đường huyết lúc đói",
             "ADVISE_ROUTINE": "duy trì lối sống lành mạnh và khám định kỳ", "REFER_DOCTOR": "nên gặp bác sĩ để được đánh giá và xác
nhận"}

class AdvisorAgent(Agent):
    tools = ("compose",)
    def cite(self, st, doc_id):
        for i, e in enumerate(st["evidence"], 1):
            if e["doc_id"] == doc_id:
                return f"[{i}]"
        return ""

    def facts(self, st, limit=3):
        return [f"- {e['text']} [{i}]" for i, e in enumerate(st["evidence"], 1)][:limit]

    def handle(self, msg):
        st = msg.payload["state"]; L = []
        level = st.get("safety", {}).get("level", "ok")
        if st.get("oos"):
            L = ["Câu hỏi này nằm ngoài phạm vi hệ thống. Hệ thống chỉ hỗ trợ thông tin về đái tháo đường."]
        elif st.get("ask"):
            L = [st["ask"]]
        elif level == "emergency":
            L = ["**Đây có thể là tình huống khẩn cấp. Hãy gọi cấp cứu 115 hoặc đến cơ sở y tế gần nhất ngay.** Không chờ hệ thống tư vấn
thêm."] + self.facts(st, 2)
        elif level == "urgent":
            L = ["**Chỉ số này có thể là hạ đường huyết, cần xử trí ngay.** Nếu không tự xử trí được hoặc mất ý thức, hãy gọi cấp cứu
115."] + self.facts(st, 3)
        elif level == "boundary":
            L = ["Hệ thống không kê đơn, không tính liều và không hướng dẫn tự thay đổi thuốc. Hãy hỏi bác sĩ điều trị trước khi thay
đổi."] + self.facts(st, 2)
        elif st.get("prediction") or st.get("decision"):
            L = self.assessment(st)
        elif st.get("plan"):
            L = []
        elif st["evidence"]:
            L = ["Theo các nguồn đã truy xuất:"] + self.facts(st, 3)
        else:
            L = ["Chưa truy xuất được bằng chứng phù hợp nên hệ thống không trả lời nội dung y khoa này. Hãy hỏi nhân viên y tế."]
        if st.get("kb_failed"):
            L.append("Lưu ý: chưa truy xuất được nguồn tri thức nên phần giải thích chưa được kiểm chứng.")
        if st.get("stale"):
            L.append(f"Lưu ý: nguồn {', '.join(st['stale'])} đã cũ, cần kiểm tra lại.")
        if st.get("plan"):
            L.append(st["plan"])
        if not st.get("oos") and not st.get("ask"):
            L.append(DISCLAIMER)
        return self.reply(msg, "DRAFT", text="\n\n".join(L))

    def assessment(self, st):
        pr, de, p = st.get("prediction"), st["decision"], st["profile"]
        L = []
        if pr:
            L.append(f"**Phân loại (DiaNet, {SCEN_VI[pr['scenario']].lower()}):** xác suất ước lượng {pct(pr['p'])}, ngưỡng phân loại
{vn(pr['threshold'] * 100)}%, nhóm {'nguy cơ cao' if pr['label'] else 'dưới ngưỡng phân loại'}.")
            L.append(f"**Dự đoán (RiskNet):** điểm nguy cơ tham khảo {vn(pr['score'])}/100, mức {pr['tier']}. Điểm này không phải xác
suất mắc bệnh.")
            if pr["drivers"]:
                L.append("Yếu tố ảnh hưởng lớn trong mô hình: " + "; ".join(f"{n} ({'+' if d >= 0 else '-'}{vn(abs(d) * 100)} điểm phần
trăm)" for n, d in pr["drivers"]) + ".")
        for c in de["lab_checks"]:
            c_ref = self.cite(st, c["doc_id"])
            word = "đạt ngưỡng:" if c["diagnostic"] else "thuộc nhóm:"
            L.append(f"**Đối chiếu:** {c['test']} {vn(c['value'], 1 if c['unit'] == '%' else 0)}{'' if c['unit'] == '%' else ' '}
{c['unit']} {word} {c['category']} {c_ref}.")
        if pr and not pr["label"] and any(c["diagnostic"] for c in de["lab_checks"]):
            L.append("Dữ liệu huấn luyện có nhãn không khớp ngưỡng y khoa ở nhóm này, nên xác suất mô hình thấp không có nghĩa là an
toàn. Hệ thống ưu tiên ngưỡng từ tài liệu.")
        rule_note = f" (luật {de['rules']} ghi đè đề xuất {de['rl_action']} của RL)" if de["rules"] else ""
```

Hình 4.23 Cell [28] Advisor, Support và Coordinator (phần 1/4).

```
        L.append(f"**Bước tiếp theo:** {ACTION_VI[de['action']]}{rule_note}.")
        tail = {"REFER_DOCTOR": "N05", "ORDER_HBA1C": "N06", "ORDER_GLUCOSE": "N06", "ADVISE_ROUTINE": "N07"}[de["action"]]
        if ref := self.cite(st, tail):
            L.append(f"{DOC[tail]['text']} {ref}")
        for doc_id, cond in (("N28", p.get("smoking") == "current"), ("N10", p.get("bmi", 0) >= 25), ("N26", p.get("activity") ==
"Low")):
            if cond and (ref := self.cite(st, doc_id)):
                L.append(f"{DOC[doc_id]['text']} {ref}")
        if pr and pr["what_if"]:
            L.append("Mô phỏng RiskNet nếu thay đổi: " + "; ".join(f"{n}: {'+' if d >= 0 else '-'}{vn(abs(d))} điểm" for n, d in
pr["what_if"]) + ". Đây là tương quan trong dữ liệu, không phải hiệu quả điều trị.")
        return L

class SupportAgent(Agent):
    tools = ("make_checklist",)
    def handle(self, msg):
        st = msg.payload["state"]; p = st.get("profile", {})
        cite = lambda d: next((f" [{i}]" for i, e in enumerate(st["evidence"], 1) if e["doc_id"] == d), "")
        labs = []
        if "hba1c" in p: labs.append(f"HbA1c {vn(p['hba1c'])}%")
        if "glucose" in p: labs.append(f"đường huyết {vn(p['glucose'], 0)} mg/dL")
        L = ["**Danh sách chuẩn bị đi khám**",
             f"- Mang kết quả đã có: {', '.join(labs) if labs else 'các xét nghiệm gần nhất nếu có'}; toa thuốc đang dùng.",
             f"- Hỏi bác sĩ có cần xét nghiệm xác nhận không{cite('N05')}.",
             f"- Hỏi mục tiêu HbA1c và huyết áp của bạn{cite('N20')}.",
             f"- Hỏi lịch khám mắt và kiểm tra bàn chân{cite('N30')}{cite('N31')}.",
             "- Ghi lại triệu chứng và câu hỏi của bạn. Hệ thống không tự đặt lịch hẹn."]
        return self.reply(msg, "PLAN", text="\n".join(L))

class Coordinator(Agent):
    """Điều phối bằng trạng thái dùng chung; mọi lời gọi agent đều đi qua call()."""
    def __init__(self, policy_mode="rl+rule", safety=True, use_dl=True):
        super().__init__("coordinator")
        self.team = {"intake": IntakeAgent("intake"), "safety": SafetyAgent("safety"), "prediction": PredictionAgent("prediction"),
                     "policy": PolicyAgent("policy", policy_mode), "knowledge": KnowledgeAgent("knowledge"),
                     "advisor": AdvisorAgent("advisor"), "support": SupportAgent("support")}
        self.safety_on, self.use_dl = safety, use_dl
        self.sessions = defaultdict(dict)

    def call(self, st, target, kind, payload, fallback=None, retries=1):
        msg = Message(self.name, target, kind, payload, st["conversation"])
        validate_message(msg)
        for attempt in range(retries + 1):
            t = time.perf_counter()
            try:
                out = validate_message(self.team[target].handle(msg))
                st["trace"].append(dict(agent=target, kind=kind, reply=out.kind, ms=round((time.perf_counter() - t) * 1000, 1),
status="ok"))
                st["messages"] += 2
                st["path"].append(target)
                st["tools"] += list(self.team[target].tools) if target != "knowledge" else out.payload["tools"]
                return out.payload
            except (TimeoutError, TypeError, ValueError, KeyError) as err:
                st["trace"].append(dict(agent=target, kind=kind, reply="ERROR", ms=round((time.perf_counter() - t) * 1000, 1), status=f"
{type(err).__name__}: {err}"))
                st["messages"] += 1; st["failures"] += 1
        return fallback

    def retrieve(self, st, queries, doc_ids=()):
        res = self.call(st, "knowledge", "RETRIEVE", {"queries": list(queries), "doc_ids": list(doc_ids)})
        if res is None:
            st["kb_failed"] = True
            return
        have = {e["doc_id"] for e in st["evidence"]}
        st["evidence"] += [e for e in res["items"] if e["doc_id"] not in have]
        st["facts"] = list(dict.fromkeys(st["facts"] + res["facts"]))
        st["stale"] = sorted(set(st["stale"]) | set(res["stale"]))
        st["expanded"] = st["expanded"] or res["expanded"]
```

Hình 4.24 Cell [28] Advisor, Support và Coordinator (phần 2/4).

```
    def ask(self, text, session="default"):
        t0 = time.perf_counter()
        memory = self.sessions[session]
        st = dict(text=text, conversation=session, trace=[], path=[], tools=[], messages=0, failures=0, evidence=[], facts=[], stale=[],
expanded=False,
                  prediction=None, decision=None, ask=None, oos=False, plan=None, safety={"level": "ok", "reason": ""}, critic_rounds=0,
issues=[])
        parsed = self.call(st, "intake", "PARSE", {"text": text, "memory": memory},
                           fallback=dict(profile=memory.get("profile", {}), intent="knowledge", errors=[], missing=[], new_fields=[]))
        st["profile"], st["intent"] = parsed["profile"], parsed["intent"]
        memory["profile"] = dict(parsed["profile"])
        if self.safety_on:
            st["safety"] = self.call(st, "safety", "PRECHECK", {"text": text, "profile": st["profile"]}, fallback=dict(level="boundary",
reason="safety unavailable"))
        level = st["safety"]["level"]

        if level == "emergency":
            self.retrieve(st, ["hạ đường huyết nặng mất ý thức cấp cứu", "nhiễm toan ceton cấp cứu"])
        elif level == "urgent":
            self.retrieve(st, ["xử trí hạ đường huyết nhẹ đo lại sau 15 phút", "hạ đường huyết nặng"], doc_ids=["N18"])
        elif level == "boundary":
            self.retrieve(st, [text, "dùng thuốc đúng chỉ định không tự đổi thuốc"])
        elif st["intent"] == "out_of_scope":
            st["oos"] = True
        elif st["intent"] == "assess":
            self.assess(st, parsed)
        elif st["intent"] == "support":
            self.checklist(st)
        else:
            self.retrieve(st, [text])
        self.write(st)
        memory["pending"] = "assess" if st["ask"] else None
        st["latency_ms"] = (time.perf_counter() - t0) * 1000
        st["answer"] = st["draft"]
        return st

    def checklist(self, st):
        self.retrieve(st, ["xét nghiệm xác nhận chẩn đoán", "mục tiêu HbA1c huyết áp"], doc_ids=["N05", "N20", "N30", "N31"])
        st["plan"] = (self.call(st, "support", "CHECKLIST", {"state": st}) or {}).get("text")

    def assess(self, st, parsed):
        if parsed["errors"] or parsed["missing"]:
            need = {"age": "tuổi", "bmi": "BMI hoặc chiều cao và cân nặng"}
            st["ask"] = ("; ".join(parsed["errors"]) + ". Hãy kiểm tra và nhập lại.") if parsed["errors"] else "Để đánh giá, hãy cho biết
thêm: " + ", ".join(need[m] for m in parsed["missing"]) + "."
            return
        p = st["profile"]
        pred = self.call(st, "prediction", "PREDICT", {"profile": p}) if self.use_dl else None
        st["prediction"] = pred
        st["decision"] = self.call(st, "policy", "DECIDE", {"p": pred["p"] if pred else .5, "profile": p},
                                   fallback=dict(action="REFER_DOCTOR", rl_action="REFER_DOCTOR", rules="FALLBACK",
lab_checks=lab_checks(p)))
        action = st["decision"]["action"]
        topic = {"REFER_DOCTOR": "cần xét nghiệm xác nhận chẩn đoán", "ADVISE_ROUTINE": "xét nghiệm lại sau mỗi 3 năm tần suất",
                 "ORDER_HBA1C": "ai nên xét nghiệm từ 35 tuổi", "ORDER_GLUCOSE": "ai nên xét nghiệm từ 35 tuổi"}[action]
        queries = [topic]
        if p.get("smoking") == "current": queries.append("hút thuốc đái tháo đường")
        if p.get("bmi", 0) >= 25: queries.append("giảm 5 đến 7 phần trăm cân nặng phòng type 2")
        if p.get("activity") == "Low": queries.append("150 phút hoạt động thể lực mỗi tuần")
        self.retrieve(st, queries)
        if action == "REFER_DOCTOR":
            self.checklist(st)

    def write(self, st):
        st["needs_citation"] = not (st["oos"] or st["ask"] or st.get("kb_failed"))
        st["draft"] = self.call(st, "advisor", "COMPOSE", {"state": st}, fallback=dict(text=DISCLAIMER))["text"]
        if not self.safety_on:
            return
        for round_ in range(3):
```

Hình 4.25 Cell [28] Advisor, Support và Coordinator (phần 3/4).

```
            verdict = self.call(st, "safety", "REVIEW", {"state": st}, fallback=dict(passed=False, issues=[("SAFETY", "unavailable")]))
            st["issues"] += verdict["issues"]
            if verdict["passed"]:
                return
            if round_ == 2:
                break
            st["critic_rounds"] += 1
            need = [v for k, v in verdict["issues"] if k == "NEED_EVIDENCE"]
            if need:
                self.retrieve(st, [DOC[d]["title"] for d in need], doc_ids=need)
            st["draft"] = self.call(st, "advisor", "COMPOSE", {"state": st}, fallback=dict(text=DISCLAIMER))["text"]
        st["draft"] = "Hệ thống không thể kiểm chứng câu trả lời nên không hiển thị nội dung chi tiết. Hãy hỏi nhân viên y tế. " +
DISCLAIMER
        st["blocked"] = True

r = Coordinator().ask("Nam 52 tuổi, BMI 27, HbA1c 6,7%, đường huyết 140 mg/dL, không hút thuốc", session="thu")
print(" → ".join(r["path"])); print(r["answer"])
```

```
intake → safety → prediction → policy → knowledge → knowledge → support → advisor → safety → knowledge → advisor → safety
**Phân loại (DiaNet, đủ hai xét nghiệm):** xác suất ước lượng 48,4%, ngưỡng phân loại 39,7%, nhóm nguy cơ cao.

**Dự đoán (RiskNet):** điểm nguy cơ tham khảo 68,3/100, mức Moderate. Điểm này không phải xác suất mắc bệnh.

Yếu tố ảnh hưởng lớn trong mô hình: Xét nghiệm HbA1c (+41,0 điểm phần trăm); Tuổi (+29,0 điểm phần trăm); Đường huyết (+11,3 điểm phần tră
m).

**Đối chiếu:** HbA1c 6,7% đạt ngưỡng: Ngưỡng đái tháo đường [8].

**Đối chiếu:** Đường huyết 140 mg/dL thuộc nhóm: Chưa rõ đo lúc đói hay không [12].

**Bước tiếp theo:** nên gặp bác sĩ để được đánh giá và xác nhận.

Bác sĩ thường dùng xét nghiệm thứ hai để xác nhận đái tháo đường. Không nên tự chẩn đoán; máy đo đường huyết mua tại cửa hàng không dùng để
chẩn đoán. [1]

Giảm 5 đến 7 phần trăm cân nặng ban đầu và duy trì có thể giúp phòng hoặc trì hoãn type 2. Nguồn dẫn nghiên cứu Diabetes Prevention Progra
m. [4]

Mô phỏng RiskNet nếu thay đổi: giảm BMI về 24: −1,2 điểm. Đây là tương quan trong dữ liệu, không phải hiệu quả điều trị.

**Danh sách chuẩn bị đi khám**
- Mang kết quả đã có: HbA1c 6,7%, đường huyết 140 mg/dL; toa thuốc đang dùng.
- Hỏi bác sĩ có cần xét nghiệm xác nhận không [1].
- Hỏi mục tiêu HbA1c và huyết áp của bạn [7].
- Hỏi lịch khám mắt và kiểm tra bàn chân [10] [11].
- Ghi lại triệu chứng và câu hỏi của bạn. Hệ thống không tự đặt lịch hẹn.

Thông tin chỉ để tham khảo, không thay thế chẩn đoán của bác sĩ.
```

Hình 4.26 Cell [28] Advisor, Support và Coordinator (phần 4/4).

CHƯƠNG 5 CODE VÀ DEMO

5.1 Tổ chức notebook và cách chạy

Toàn bộ mã nguồn nằm trong một notebook có mục lục theo sơ đồ trên bảng. Chạy tuần tự từ đầu đến cuối mất khoảng 2 phút trên CPU, kể cả huấn luyện lại mô hình. Môi trường riêng đặt ở thư mục .venv trên ổ D, kernel có tên Python (project01). Bấm đúp tệp start_notebook.bat để mở JupyterLab với kernel này, hoặc trong VS Code chọn interpreter .venv. Các lệnh trong tệp:

```
set JUPYTER_PATH=%~dp0.venv\share\jupyter
jupyter lab project01_05_15_haih.ipynb
```

Bảng 5.1 Các phần của notebook

Mục	Nội dung	Cell
0–1	Cấu hình, dữ liệu	[1]–[3]
2–3	DiaNet, RiskNet	[4]–[12]
4	KG và RAG	[13]–[18]
5	Q-learning và Rule	[19]–[22]
6	Kiến trúc và các agent	[23]–[28]
7	Demo 7 truy vấn	[29]–[33]
8	Kiểm thử và tiêm lỗi	[34]
9	Đánh giá hệ thống	[35]–[37]
10	Demo tương tác	[38]–[39]

In [1]:
```
import os
os.environ.setdefault("OMP_NUM_THREADS", "1")  # tránh xung đột OpenMP giữa PyTorch và scikit-learn trên Windows
import json, re, time, uuid, unicodedata, warnings, platform
from collections import Counter, defaultdict
from dataclasses import dataclass, field, asdict
from pathlib import Path

import matplotlib.pyplot as plt
import networkx as nx
import numpy as np
import pandas as pd
import sklearn
import torch
from IPython.display import display, Markdown
from sklearn.decomposition import TruncatedSVD
from sklearn.ensemble import HistGradientBoostingClassifier, HistGradientBoostingRegressor
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score, average_precision_score, brier_score_loss, confusion_matrix, f1_score,
                             mean_absolute_error, precision_recall_curve, precision_score, r2_score, recall_score,
                             roc_auc_score)
from sklearn.model_selection import train_test_split
from torch import nn

warnings.filterwarnings("ignore")
SEED = 42
ROOT = Path.cwd()
DATA, RES = ROOT / "data", ROOT / "results"
FIG = RES / "figures"
FIG.mkdir(parents=True, exist_ok=True)
torch.set_num_threads(1)  # máy 8 GB RAM, mạng nhỏ nên một luồng là đủ và ổn định hơn

BLUE, ORANGE, AQUA, YELLOW, VIOLET, GRAY = "#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#6250d6", "#8a8984"
plt.rcParams.update({"figure.dpi": 110, "savefig.dpi": 170, "font.size": 10, "axes.spines.top": False,
                     "axes.spines.right": False, "axes.grid": True, "grid.color": "#e8e7e3", "grid.linewidth": .7,
                     "axes.axisbelow": True, "legend.frameon": False, "figure.facecolor": "white"})

def set_seed(seed):
    np.random.seed(seed); torch.manual_seed(seed)

def save(fig, name):
    fig.tight_layout(); fig.savefig(FIG / f"{name}.png", bbox_inches="tight"); plt.show()

print("Python", platform.python_version(), "| PyTorch", torch.__version__, "| scikit-learn", sklearn.__version__,
      "| NumPy", np.__version__, "| pandas", pd.__version__)
```

```
Python 3.13.7 | PyTorch 2.15.0+cpu | scikit-learn 1.9.0 | NumPy 2.5.2 | pandas 3.0.5
```

Hình 5.1 Cell [1] cấu hình môi trường.

5.2 Demo các truy vấn

Bài giảng yêu cầu ít nhất năm truy vấn, mỗi truy vấn nêu câu hỏi, luồng agent, công cụ, thông tin KG, tài liệu, câu trả lời, bằng chứng và đánh giá [20]. Hàm report() trong notebook in đủ các mục này. Ảnh dưới đây là output thật của notebook.

Truy vấn 1: có xét nghiệm, đạt ngưỡng

Hồ sơ có HbA1c 7,5%. Hệ thống đi qua 24 thông điệp và một vòng Critic: bản nháp đầu thiếu tài liệu ngưỡng, Safety yêu cầu Knowledge lấy thêm rồi Advisor viết lại. Support tự được gọi vì quyết định là chuyển khám.

In [29]:

```
def report(r, title=""):
    ev = "\n".join(f"{i}. **{e['doc_id']}** ({e['source_type']}) {e['title']}: {e['publisher']}{', ' + e['locator'] if e['locator'] else ''}" for i, e in enumerate(r["evidence"], 1)) or "Không có"
    kg = "\n".join(f"- {f}" for f in r["facts"][:4]) or "Không có"
    tools = ", ".join(dict.fromkeys(r["tools"]))
    de = r.get("decision") or {}
    quick = f"mức an toàn `{r['safety']['level']}`, vòng Critic {r['critic_rounds']}, tin nhắn {r['messages']}, lỗi {r['failures']}, thời gian {r['latency_ms']:.0f} ms"
    if de: quick += f", RL đề xuất `{de['rl_action']}`, quyết định `{de['action']}`" + (f", luật `{de['rules']}`" if de["rules"] else "")
    display(Markdown(f"""### {title}
**1. Câu hỏi:** {r['text']}

**2. Luồng agent:** {' → '.join(r['path'])}

**3. Công cụ:** {tools}

**4. Thông tin KG:**
{kg}

**5. Tài liệu truy xuất và bằng chứng:**
{ev}

**6. Câu trả lời:**

{r['answer']}

**7. Đánh giá nhanh:** {quick}
"""))

DEMO = {}
demo = Coordinator()
DEMO["q1"] = demo.ask("Nam 58 tuổi, BMI 33,5, tăng huyết áp, HbA1c 7,5%, đường huyết 180 mg/dL. Nguy cơ của tôi thế nào?", "d1")
report(DEMO["q1"], "Truy vấn 1: có xét nghiệm, đạt ngưỡng")
```

Truy vấn 1: có xét nghiệm, đạt ngưỡng

1. Câu hỏi: Nam 58 tuổi, BMI 33,5, tăng huyết áp, HbA1c 7,5%, đường huyết 180 mg/dL. Nguy cơ của tôi thế nào?

2. Luồng agent: intake → safety → prediction → policy → knowledge → knowledge → support → advisor → safety → knowledge → advisor → safety

3. Công cụ: parse_profile, red_flags, critic, DiaNet, RiskNet, Q-table, threshold_lookup, search_kb, make_checklist, compose

4. Thông tin KG:

- (Chẩn đoán đái tháo đường, cần, Xét nghiệm xác nhận) [N05]
- (Type 2, có_thể_cần, Insulin) [M02]
- (Type 2, là_dạng, Đái tháo đường) [X01]
- (Type 2, có_thể_tiến_triển, Nhiều năm) [X02]

5. Tài liệu truy xuất và bằng chứng:

1. **N05** (web) Cần xét nghiệm xác nhận: NIDDK
2. **N07** (web) Tần suất xét nghiệm lại: NIDDK
3. **X06** (web) Xét nghiệm dùng trong chẩn đoán: NIDDK
4. **N10** (web) Giảm cân để phòng type 2: NIDDK
5. **N12** (web) Ăn uống để phòng type 2: NIDDK
6. **N35** (web) Phòng ngừa theo WHO: WHO
7. **N20** (web) Mục tiêu ABC: NIDDK
8. **N01** (web) Ngưỡng HbA1c: NIDDK
9. **N09** (web) Khi HbA1c có thể không chính xác: NIDDK
10. **N30** (web) Biến chứng mắt: NIDDK
11. **N31** (web) Biến chứng bàn chân: NIDDK
12. **N02** (web) Ngưỡng đường huyết lúc đói: NIDDK

Hình 5.2 Cell [29] hàm report() và truy vấn 1 (phần 1/2).

13. **N04** (web) Đường huyết ngẫu nhiên: NIDDK
14. **N03** (web) Ngưỡng nghiệm pháp dung nạp glucose: NIDDK

6. Câu trả lời:

Phân loại (DiaNet, đủ hai xét nghiệm): xác suất ước lượng trên 99,9%, ngưỡng phân loại 39,7%, nhóm nguy cơ cao.

Dự đoán (RiskNet): điểm nguy cơ tham khảo 85,7/100, mức High. Điểm này không phải xác suất mắc bệnh.

Yếu tố ảnh hưởng lớn trong mô hình: Xét nghiệm HbA1c (+82,5 điểm phần trăm).

Đối chiếu: HbA1c 7,5% đạt ngưỡng: Ngưỡng đái tháo đường [8].

Đối chiếu: Đường huyết 180 mg/dL thuộc nhóm: Chưa rõ đo lúc đói hay không [12].

Bước tiếp theo: nên gặp bác sĩ để được đánh giá và xác nhận.

Bác sĩ thường dùng xét nghiệm thứ hai để xác nhận đái tháo đường. Không nên tự chẩn đoán; máy đo đường huyết mua tại cửa hàng không dùng để chẩn đoán. [1]

Giảm 5 đến 7 phần trăm cân nặng ban đầu và duy trì có thể giúp phòng hoặc trì hoãn type 2. Nguồn dẫn nghiên cứu Diabetes Prevention Program. [4]

Mô phỏng RiskNet nếu thay đổi: giảm BMI về 24: −5,2 điểm. Đây là tương quan trong dữ liệu, không phải hiệu quả điều trị.

Danh sách chuẩn bị đi khám

- Mang kết quả đã có: HbA1c 7,5%, đường huyết 180 mg/dL; toa thuốc đang dùng.
- Hỏi bác sĩ có cần xét nghiệm xác nhận không [1].
- Hỏi mục tiêu HbA1c và huyết áp của bạn [7].
- Hỏi lịch khám mắt và kiểm tra bàn chân [10] [11].
- Ghi lại triệu chứng và câu hỏi của bạn. Hệ thống không tự đặt lịch hẹn.

Thông tin chỉ để tham khảo, không thay thế chẩn đoán của bác sĩ.

7. Đánh giá nhanh: mức an toàn `ok`, vòng Critic 1, tin nhắn 24, lỗi 0, thời gian 27 ms, RL đề xuất `REFER_DOCTOR`, quyết định `REFER_DOCTOR`

Hình 5.3 Cell [29] hàm report() và truy vấn 1 (phần 2/2).

Truy vấn 2: mô hình và Rule mâu thuẫn

Đây là ca thể hiện rõ nhất vì sao cần Rule. HbA1c 6,6% nhưng đường huyết 100 mg/dL nên DiaNet báo xác suất dưới 0,1% và Q-learning đề xuất ADVISE_ROUTINE. Luật R1 ghi đè thành REFER_DOCTOR, Advisor giải thích mâu thuẫn giữa dữ liệu và ngưỡng y khoa.

```
In [30]: DEMO["q2"] = demo.ask("Nam 35 tuổi, BMI 24, HbA1c 6,6%, đường huyết 100 mg/dL, không hút thuốc", "d2")
         report(DEMO["q2"], "Truy vấn 2: mô hình và Rule mâu thuẫn")
```

Truy vấn 2: mô hình và Rule mâu thuẫn

1. Câu hỏi: Nam 35 tuổi, BMI 24, HbA1c 6,6%, đường huyết 100 mg/dL, không hút thuốc

2. Luồng agent: intake → safety → prediction → policy → knowledge → knowledge → support → advisor → safety

3. Công cụ: parse_profile, red_flags, critic, DiaNet, RiskNet, Q-table, threshold_lookup, search_kb, make_checklist, compose

4. Thông tin KG:

- (Chẩn đoán đái tháo đường, cần, Xét nghiệm xác nhận) [N05]
- (Quản lý Type 2, bao_gồm_kiểm_soát, Huyết áp) [X04]
- (HbA1c, hỗ_trợ_chẩn_đoán, Đái tháo đường) [X06]
- (HbA1c, phản_ánh, Đường huyết trung bình khoảng 3 tháng) [X06]

5. Tài liệu truy xuất và bằng chứng:

1. **N05** (web) Cần xét nghiệm xác nhận: NIDDK
2. **N07** (web) Tần suất xét nghiệm lại: NIDDK
3. **X06** (web) Xét nghiệm dùng trong chẩn đoán: NIDDK
4. **N20** (web) Mục tiêu ABC: NIDDK
5. **N01** (web) Ngưỡng HbA1c: NIDDK
6. **N09** (web) Khi HbA1c có thể không chính xác: NIDDK
7. **N30** (web) Biến chứng mắt: NIDDK
8. **N31** (web) Biến chứng bàn chân: NIDDK

6. Câu trả lời:

Phân loại (DiaNet, đủ hai xét nghiệm): xác suất ước lượng dưới 0,1%, ngưỡng phân loại 39,7%, nhóm dưới ngưỡng phân loại.

Dự đoán (RiskNet): điểm nguy cơ tham khảo 60,6/100, mức Moderate. Điểm này không phải xác suất mắc bệnh.

Yếu tố ảnh hưởng lớn trong mô hình: Đường huyết (−3,0 điểm phần trăm).

Đối chiếu: HbA1c 6,6% đạt ngưỡng: Ngưỡng đái tháo đường [5].

Dữ liệu huấn luyện có nhãn không khớp ngưỡng y khoa ở nhóm này, nên xác suất mô hình thấp không có nghĩa là an toàn. Hệ thống ưu tiên ngưỡng từ tài liệu.

Bước tiếp theo: nên gặp bác sĩ để được đánh giá và xác nhận (luật R1 ghi đè đề xuất ADVISE_ROUTINE của RL).

Bác sĩ thường dùng xét nghiệm thứ hai để xác nhận đái tháo đường. Không nên tự chẩn đoán; máy đo đường huyết mua tại cửa hàng không dùng để chẩn đoán. [1]

Danh sách chuẩn bị đi khám

- Mang kết quả đã có: HbA1c 6,6%, đường huyết 100 mg/dL; toa thuốc đang dùng.
- Hỏi bác sĩ có cần xét nghiệm xác nhận không [1].
- Hỏi mục tiêu HbA1c và huyết áp của bạn [4].
- Hỏi lịch khám mắt và kiểm tra bàn chân [7] [8].
- Ghi lại triệu chứng và câu hỏi của bạn. Hệ thống không tự đặt lịch hẹn.

Thông tin chỉ để tham khảo, không thay thế chẩn đoán của bác sĩ.

7. Đánh giá nhanh: mức an toàn `ok`, vòng Critic 0, tin nhắn 18, lỗi 0, thời gian 18 ms, RL đề xuất `ADVISE_ROUTINE`, quyết định `REFER_DOCTOR`, luật `R1`

Hình 5.4 Cell [30] truy vấn 2.

Truy vấn 3: nhiều lượt và học từ thông tin mới

Lượt đầu chưa có xét nghiệm, xác suất 25,2%, Policy Agent yêu cầu đo đường huyết lúc đói. Lượt sau người dùng chỉ gửi kết quả xét nghiệm, hệ thống dùng bộ nhớ để ghép hồ sơ, tính lại và chuyển sang tư vấn định kỳ.

In [31]:
```
DEMO["q3a"] = demo.ask("Nam 50 tuổi, BMI 29, tăng huyết áp, đang hút thuốc. Tôi có nguy cơ tiểu đường không?", "d3")
report(DEMO["q3a"], "Truy vấn 3a: chưa có xét nghiệm, Policy Agent yêu cầu thêm dữ liệu")
DEMO["q3b"] = demo.ask("Tôi vừa có kết quả: HbA1c 5,4%, đường huyết lúc đói 96 mg/dL", "d3")
report(DEMO["q3b"], "Truy vấn 3b: lượt sau dùng bộ nhớ và kết quả xét nghiệm mới")
```

Truy vấn 3a: chưa có xét nghiệm, Policy Agent yêu cầu thêm dữ liệu

1. Câu hỏi: Nam 50 tuổi, BMI 29, tăng huyết áp, đang hút thuốc. Tôi có nguy cơ tiểu đường không?

2. Luồng agent: intake → safety → prediction → policy → knowledge → advisor → safety

3. Công cụ: parse_profile, red_flags, critic, DiaNet, RiskNet, Q-table, threshold_lookup, search_kb, compose

4. Thông tin KG:

- (Đái tháo đường, đặc_trưng_bởi, Tăng đường huyết) [X01]
- (Đái tháo đường, có_thể_có_triệu_chứng, Khát nhiều) [X02]
- (Đái tháo đường, có_thể_có_triệu_chứng, Tiểu nhiều) [X02]
- (Hút thuốc, tăng_nguy_cơ, Bệnh tim) [N28]

5. Tài liệu truy xuất và bằng chứng:

1. **N06** (web) Ai nên xét nghiệm: NIDDK
2. **N07** (web) Tần suất xét nghiệm lại: NIDDK
3. **N05** (web) Cần xét nghiệm xác nhận: NIDDK
4. **N28** (web) Hút thuốc: NIDDK
5. **N32** (web) Tim mạch và đột quỵ: NIDDK
6. **N01** (web) Ngưỡng HbA1c: NIDDK
7. **N10** (web) Giảm cân để phòng type 2: NIDDK
8. **N12** (web) Ăn uống để phòng type 2: NIDDK
9. **N35** (web) Phòng ngừa theo WHO: WHO

6. Câu trả lời:

Phân loại (DiaNet, chưa xét nghiệm): xác suất ước lượng 25,2%, ngưỡng phân loại 20,2%, nhóm nguy cơ cao.

Dự đoán (RiskNet): điểm nguy cơ tham khảo 82,6/100, mức High. Điểm này không phải xác suất mắc bệnh.

Yếu tố ảnh hưởng lớn trong mô hình: Tăng huyết áp (+15,0 điểm phần trăm); Tuổi (+14,8 điểm phần trăm); BMI (+6,0 điểm phần trăm).

Bước tiếp theo: nên đo đường huyết lúc đói.

Người có triệu chứng cần được xét nghiệm. Nên xét nghiệm định kỳ type 2 nếu từ 35 tuổi trở lên, thừa cân hoặc béo phì kèm thêm yếu tố nguy cơ, hoặc từng mắc đái tháo đường thai kỳ. [1]

Đái tháo đường và hút thuốc đều làm hẹp mạch máu, cùng làm tăng nguy cơ đau tim, đột quỵ, tổn thương thần kinh, bệnh thận, bệnh mắt và cắt cụt chi. Người hút thuốc nên ngừng và tìm hỗ trợ. [4]

Giảm 5 đến 7 phần trăm cân nặng ban đầu và duy trì có thể giúp phòng hoặc trì hoãn type 2. Nguồn dẫn nghiên cứu Diabetes Prevention Program. [7]

Mô phỏng RiskNet nếu thay đổi: giảm BMI về 24: −2,2 điểm; ngừng hút thuốc: −7,6 điểm. Đây là tương quan trong dữ liệu, không phải hiệu quả điều trị.

Thông tin chỉ để tham khảo, không thay thế chẩn đoán của bác sĩ.

7. Đánh giá nhanh: mức an toàn `ok`, vòng Critic 0, tin nhắn 14, lỗi 0, thời gian 18 ms, RL đề xuất `ORDER_GLUCOSE`, quyết định `ORDER_GLUCOSE`

Truy vấn 3b: lượt sau dùng bộ nhớ và kết quả xét nghiệm mới

1. Câu hỏi: Tôi vừa có kết quả: HbA1c 5,4%, đường huyết lúc đói 96 mg/dL

2. Luồng agent: intake → safety → prediction → policy → knowledge → advisor → safety → knowledge → advisor → safety

Hình 5.5 Cell [31] truy vấn 3a và 3b (phần 1/2).

3. Công cụ: parse_profile, red_flags, critic, DiaNet, RiskNet, Q-table, threshold_lookup, search_kb, compose

4. Thông tin KG:

- (Đái tháo đường, đặc_trưng_bởi, Tăng đường huyết) [X01]
- (Đái tháo đường, có_thể_có_triệu_chứng, Khát nhiều) [X02]
- (Đái tháo đường, có_thể_có_triệu_chứng, Tiểu nhiều) [X02]
- (Hút thuốc, tăng_nguy_cơ, Bệnh tim) [N28]

5. Tài liệu truy xuất và bằng chứng:

1. **N07** (web) Tần suất xét nghiệm lại: NIDDK
2. **N08** (web) Xét nghiệm khi mang thai: NIDDK
3. **N06** (web) Ai nên xét nghiệm: NIDDK
4. **N28** (web) Hút thuốc: NIDDK
5. **N32** (web) Tim mạch và đột quỵ: NIDDK
6. **N01** (web) Ngưỡng HbA1c: NIDDK
7. **N10** (web) Giảm cân để phòng type 2: NIDDK
8. **N12** (web) Ăn uống để phòng type 2: NIDDK
9. **N35** (web) Phòng ngừa theo WHO: WHO
10. **N02** (web) Ngưỡng đường huyết lúc đói: NIDDK
11. **N04** (web) Đường huyết ngẫu nhiên: NIDDK
12. **N03** (web) Ngưỡng nghiệm pháp dung nạp glucose: NIDDK

6. Câu trả lời:

Phân loại (DiaNet, đủ hai xét nghiệm): xác suất ước lượng dưới 0,1%, ngưỡng phân loại 39,7%, nhóm dưới ngưỡng phân loại.

Dự đoán (RiskNet): điểm nguy cơ tham khảo 54,3/100, mức Moderate. Điểm này không phải xác suất mắc bệnh.

Yếu tố ảnh hưởng lớn trong mô hình: Đường huyết (−3,5 điểm phần trăm).

Đối chiếu: HbA1c 5,4% thuộc nhóm: Bình thường [6].

Đối chiếu: Đường huyết lúc đói 96 mg/dL thuộc nhóm: Bình thường [10].

Bước tiếp theo: duy trì lối sống lành mạnh và khám định kỳ.

Người có kết quả bình thường nên xét nghiệm lại sau mỗi 3 năm. Người được chẩn đoán tiền đái tháo đường nên xét nghiệm type 2 hằng năm. [1]

Đái tháo đường và hút thuốc đều làm hẹp mạch máu, cùng làm tăng nguy cơ đau tim, đột quỵ, tổn thương thần kinh, bệnh thận, bệnh mắt và cắt cụt chi. Người hút thuốc nên ngừng và tìm hỗ trợ. [4]

Giảm 5 đến 7 phần trăm cân nặng ban đầu và duy trì có thể giúp phòng hoặc trì hoãn type 2. Nguồn dẫn nghiên cứu Diabetes Prevention Program. [7]

Mô phỏng RiskNet nếu thay đổi: giảm BMI về 24: −1,8 điểm; ngừng hút thuốc: −3,6 điểm. Đây là tương quan trong dữ liệu, không phải hiệu quả điều trị.

Thông tin chỉ để tham khảo, không thay thế chẩn đoán của bác sĩ.

7. Đánh giá nhanh: mức an toàn `ok`, vòng Critic 1, tin nhắn 20, lỗi 0, thời gian 24 ms, RL đề xuất `ADVISE_ROUTINE`, quyết định `ADVISE_ROUTINE`

Hình 5.6 Cell [31] truy vấn 3a và 3b (phần 2/2).

Truy vấn 4 và 5: hỏi kiến thức, ranh giới thuốc

Câu hỏi kiến thức không đi qua Prediction và Policy. Câu đòi liều metformin bị từ chối phần kê đơn, hệ thống chỉ cung cấp thông tin chung có nguồn.

In [32]:
```
DEMO["q4"] = demo.ask("Triệu chứng của tiểu đường là gì?", "d4")
report(DEMO["q4"], "Truy vấn 4: hỏi kiến thức")
DEMO["q5"] = demo.ask("Kê cho tôi metformin liều bao nhiêu mg một ngày?", "d5")
report(DEMO["q5"], "Truy vấn 5: ranh giới thuốc")
```

Truy vấn 4: hỏi kiến thức

1. Câu hỏi: Triệu chứng của tiểu đường là gì?

2. Luồng agent: intake → safety → knowledge → advisor → safety

3. Công cụ: parse_profile, red_flags, critic, search_kb, compose

4. Thông tin KG:

- (Đái tháo đường, đặc_trưng_bởi, Tăng đường huyết) [X01]
- (Đái tháo đường, có_thể_có_triệu_chứng, Khát nhiều) [X02]
- (Đái tháo đường, có_thể_có_triệu_chứng, Tiểu nhiều) [X02]

5. Tài liệu truy xuất và bằng chứng:

1. **X02** (web) Triệu chứng và diễn tiến: NIDDK
2. **X08** (web) Biến chứng của đái tháo đường: NIDDK
3. **X01** (web) Sinh lý glucose và phân loại đái tháo đường: NIDDK

6. Câu trả lời:

Theo các nguồn đã truy xuất:

- Đái tháo đường có thể gây khát, tiểu hoặc đói nhiều, nhìn mờ, mệt, vết thương lâu lành và nhiễm trùng thường xuyên. Type 1 có thể kèm sụt cân không rõ nguyên nhân. Type 2 thường tiến triển âm thầm trong nhiều năm; triệu chứng không đủ để tự chẩn đoán. [1]
- Đái tháo đường có thể làm tổn thương mạch máu và thần kinh, dẫn tới bệnh tim, đột quỵ, bệnh thận, vấn đề bàn chân, mất hoặc nướu. Tổn thương thần kinh và máu lưu thông kém góp phần vào vấn đề bàn chân. Đây là nguy cơ, không phải biến chứng xảy ra ở mọi người. [2]
- Tụy tạo insulin, một hormone giúp glucose vào tế bào để cung cấp năng lượng. Đường huyết tăng là đặc điểm của đái tháo đường. Type 1, Type 2, thai kỳ và đơn gen là các dạng bệnh. Glucose đến từ thức ăn và cũng được cơ thể tạo ra. [3]

Thông tin chỉ để tham khảo, không thay thế chẩn đoán của bác sĩ.

7. Đánh giá nhanh: mức an toàn `ok`, vòng Critic 0, tin nhắn 10, lỗi 0, thời gian 7 ms

Truy vấn 5: ranh giới thuốc

1. Câu hỏi: Kê cho tôi metformin liều bao nhiêu mg một ngày?

2. Luồng agent: intake → safety → knowledge → advisor → safety

3. Công cụ: parse_profile, red_flags, critic, search_kb, compose

4. Thông tin KG:

- (Metformin, thuộc_nhóm, Biguanide) [M04]
- (Metformin, được_dùng_cho, Type 2) [M04]
- (Metformin, giảm, Gan tạo glucose) [X10]

5. Tài liệu truy xuất và bằng chứng:

1. **M04** (web) Metformin: UK
2. **X10** (web) Cơ chế các nhóm thuốc: Diabetes UK
3. **N13** (web) Khi có tiền đái tháo đường: NIDDK

Hình 5.7 Cell [32] truy vấn 4 và 5 (phần 1/2).

4. **N22** (web) Dùng thuốc đúng chỉ định: NIDDK
5. **M14** (web) Theo dõi khi dùng thuốc: NIDDK_MEDS
6. **M03** (web) Cá thể hóa thuốc: NIDDK_MEDS

6. Câu trả lời:

Hệ thống không kê đơn, không tính liều và không hướng dẫn tự thay đổi thuốc. Hãy hỏi bác sĩ điều trị trước khi thay đổi.

- Metformin thuộc biguanide; giảm glucose từ gan và hỗ trợ sử dụng insulin trong điều trị type 2. [1]
- Metformin làm gan tạo ít glucose hơn và giúp cơ thể dùng insulin tốt hơn. SGLT2 tăng thải glucose qua nước tiểu. Sulfonylurea và glinide kích thích tiết insulin. Alpha-glucosidase làm chậm hấp thu tinh bột. TZD tăng nhạy insulin; DPP-4 bảo vệ incretin, GLP-1 hỗ trợ tiết insulin khi cần. [2]

Thông tin chỉ để tham khảo, không thay thế chẩn đoán của bác sĩ.

7. Đánh giá nhanh: mức an toàn `boundary`, vòng Critic 0, tin nhắn 10, lỗi 0, thời gian 9 ms

Hình 5.8 Cell [32] truy vấn 4 và 5 (phần 2/2).

Truy vấn 6 và 7: cảnh báo và trợ giúp

Đường huyết 55 mg/dL kích hoạt mức urgent, hệ thống đưa cách xử trí từ nguồn và điều kiện gọi cấp cứu. Truy vấn 7 lập danh sách chuẩn bị đi khám, đồng thời ghi rõ hệ thống không tự đặt lịch.

```
In [33]: DEMO["q6"] = demo.ask("Tôi run, vã mồ hôi, đường huyết đo được 55 mg/dL thì làm sao?", "d6")
         report(DEMO["q6"], "Truy vấn 6: cảnh báo hạ đường huyết")
         DEMO["q7"] = demo.ask("Giúp tôi chuẩn bị đi khám tiểu đường, cần mang gì và hỏi bác sĩ gì?", "d7")
         report(DEMO["q7"], "Truy vấn 7: trợ giúp chuẩn bị đi khám")
         pd.DataFrame(DEMO["q1"]["trace"]).head(12)
```

Truy vấn 6: cảnh báo hạ đường huyết

1. Câu hỏi: Tôi run, vã mồ hôi, đường huyết đo được 55 mg/dL thì làm sao?

2. Luồng agent: intake → safety → knowledge → advisor → safety

3. Công cụ: parse_profile, red_flags, critic, search_kb, compose

4. Thông tin KG:

- (Hạ đường huyết, có_thể_có_triệu_chứng, Run) [N16]
- (Hạ đường huyết, có_thể_có_triệu_chứng, Chóng mặt) [N16]
- (Hạ đường huyết, có_thể_có_triệu_chứng, Lú lẫn) [N16]
- (Quản lý Type 2, bao_gồm_kiểm_soát, Đường huyết) [X04]

5. Tài liệu truy xuất và bằng chứng:

1. **N18** (web) Xử trí hạ đường huyết nhẹ: NIDDK
2. **N19** (web) Hạ đường huyết nặng: NIDDK
3. **N16** (web) Triệu chứng hạ đường huyết: NIDDK
4. **N15** (web) Hạ đường huyết là gì: NIDDK
5. **N17** (web) Ai dễ bị hạ đường huyết: NIDDK

6. Câu trả lời:

Chỉ số này có thể là hạ đường huyết, cần xử trí ngay. Nếu không tự xử trí được hoặc mất ý thức, hãy gọi cấp cứu 115.

- Khi có triệu chứng hoặc đường huyết dưới 70 mg/dL: dùng ngay 15 đến 20 gam glucose hoặc carbohydrate, chờ 15 phút rồi đo lại, lặp lại nếu vẫn thấp. [1]
- Hạ đường huyết nặng là khi người bệnh không tự xử trí được; có thể gây ngất, hôn mê, tử vong. Glucagon là cách điều trị tốt nhất và người xung quanh cần gọi cấp cứu ngay. [2]
- Hạ đường huyết nhẹ đến vừa có thể gây run, đói, mệt, chóng mặt, lú lẫn, cáu gắt, tim đập nhanh, đau đầu. Mức rất thấp có thể gây mất ý thức hoặc co giật. [3]

Thông tin chỉ để tham khảo, không thay thế chẩn đoán của bác sĩ.

7. Đánh giá nhanh: mức an toàn `urgent`, vòng Critic 0, tin nhắn 10, lỗi 0, thời gian 9 ms

Truy vấn 7: trợ giúp chuẩn bị đi khám

1. Câu hỏi: Giúp tôi chuẩn bị đi khám tiểu đường, cần mang gì và hỏi bác sĩ gì?

2. Luồng agent: intake → safety → knowledge → support → advisor → safety

3. Công cụ: parse_profile, red_flags, critic, search_kb, make_checklist, compose

4. Thông tin KG:

- (Chẩn đoán đái tháo đường, cần, Xét nghiệm xác nhận) [N05]
- (Quản lý Type 2, bao_gồm_kiểm_soát, Huyết áp) [X04]
- (HbA1c, hỗ_trợ_chẩn_đoán, Đái tháo đường) [X06]
- (HbA1c, phản_ánh, Đường huyết trung bình khoảng 3 tháng) [X06]

5. Tài liệu truy xuất và bằng chứng:

Hình 5.9 Cell [33] truy vấn 6, 7 và vết thông điệp của truy vấn 1 (phần 1/2).

1. **N05** (web) Cần xét nghiệm xác nhận: NIDDK
2. **N07** (web) Tần suất xét nghiệm lại: NIDDK
3. **X06** (web) Xét nghiệm dùng trong chẩn đoán: NIDDK
4. **N20** (web) Mục tiêu ABC: NIDDK
5. **N01** (web) Ngưỡng HbA1c: NIDDK
6. **N09** (web) Khi HbA1c có thể không chính xác: NIDDK
7. **N30** (web) Biến chứng mắt: NIDDK
8. **N31** (web) Biến chứng bàn chân: NIDDK

6. Câu trả lời:

Danh sách chuẩn bị đi khám

- Mang kết quả đã có: các xét nghiệm gần nhất nếu có; toa thuốc đang dùng.
- Hỏi bác sĩ có cần xét nghiệm xác nhận không [1].
- Hỏi mục tiêu HbA1c và huyết áp của bạn [4].
- Hỏi lịch khám mắt và kiểm tra bàn chân [7] [8].
- Ghi lại triệu chứng và câu hỏi của bạn. Hệ thống không tự đặt lịch hẹn.

Thông tin chỉ để tham khảo, không thay thế chẩn đoán của bác sĩ.

7. Đánh giá nhanh: mức an toàn `ok`, vòng Critic 0, tin nhắn 12, lỗi 0, thời gian 7 ms

Out[33]:

	agent	kind	reply	ms	status
0	intake	PARSE	PARSED	0.9	ok
1	safety	PRECHECK	SAFETY	0.0	ok
2	prediction	PREDICT	PREDICTION	9.5	ok
3	policy	DECIDE	DECISION	1.0	ok
4	knowledge	RETRIEVE	EVIDENCE	5.1	ok
5	knowledge	RETRIEVE	EVIDENCE	4.6	ok
6	support	CHECKLIST	PLAN	0.0	ok
7	advisor	COMPOSE	DRAFT	0.0	ok
8	safety	REVIEW	VERDICT	0.2	ok
9	knowledge	RETRIEVE	EVIDENCE	2.8	ok
10	advisor	COMPOSE	DRAFT	0.1	ok
11	safety	REVIEW	VERDICT	0.5	ok

Hình 5.10 Cell [33] truy vấn 6, 7 và vết thông điệp của truy vấn 1 (phần 2/2).

Demo tương tác

Khi trình bày, có thể nhập câu hỏi trực tiếp vào ô ở cuối notebook hoặc gọi consult('câu hỏi'). Ô nhập cần kernel đang chạy nên phải chạy các cell trước đó.

In [38]:
```
import ipywidgets as w
live = Coordinator()
box = w.Textarea(placeholder="Ví dụ: Nữ 45 tuổi, BMI 26, HbA1c 6,1%, đánh giá nguy cơ", layout=w.Layout(width="100%", height="70px"))
send, reset, out = w.Button(description="Gửi", button_style="primary"), w.Button(description="Phiên mới"), w.Output()
session = {"id": "live-0", "n": 0}

def on_send(_):
    with out:
        out.clear_output()
        if box.value.strip():
            report(live.ask(box.value, session["id"]), "Kết quả")

def on_reset(_):
    session["n"] += 1; session["id"] = f"live-{session['n']}"
    with out:
        out.clear_output(); print("Đã bắt đầu phiên mới")

send.on_click(on_send); reset.on_click(on_reset)
display(w.VBox([box, w.HBox([send, reset]), out]))
```



Hình 5.11 Cell [38] giao diện nhập câu hỏi trực tiếp trong notebook.

5.3 Kiểm thử và tiêm lỗi

Bài giảng yêu cầu ít nhất 8 kiểm thử, trong đó 3 kiểm thử lỗi [20]. Notebook có 12 kiểm thử, gồm 4 kiểm thử lỗi, và tất cả đều đạt.

Bảng 5.2 Danh sách kiểm thử

Kiểm thử	Loại	Kết quả
Knowledge timeout có fallback, không bịa nguồn	Lỗi	PASS
Prediction trả sai schema, Rule vẫn chuyển khám	Lỗi	PASS
Truy vấn vô nghĩa không trả bằng chứng	Lỗi	PASS
Phát hiện nguồn cũ	Lỗi	PASS
Critic chặn bản nháp có liều, chẩn đoán, số và trích dẫn bịa	An toàn	PASS
Rule ghi đè khi HbA1c đạt ngưỡng	An toàn	PASS
RL không yêu cầu lại xét nghiệm đã biết	Biên	PASS
Đổi mmol/L sang mg/dL	Bình thường	PASS
Tính BMI từ chiều cao, cân nặng	Bình thường	PASS
Bộ nhớ ghép hồ sơ qua hai lượt	Bình thường	PASS
Schema từ chối tin nhắn thiếu trường	Biên	PASS
Xác suất DiaNet nằm trong [0, 1]	Biên	PASS

In [34]:

```
class TimeoutAgent(Agent):
    tools = ()
    def handle(self, msg):
        raise TimeoutError("knowledge timeout")

class MalformedAgent(Agent):
    tools = ()
    def handle(self, msg):
        return Message("prediction", "coordinator", "PREDICTION", {"p": .5})  # thiếu trường

class BadAdvisor(AdvisorAgent):
    def handle(self, msg):
        return self.reply(msg, "DRAFT", text="Bạn bị đái tháo đường, hãy uống metformin 500 mg, HbA1c mục tiêu 4,2% [99]")

tests = []
def test(name, kind, fn):
    try:
        ok = bool(fn()); note = ""
    except Exception as err:
        ok, note = False, repr(err)
    tests.append({"Kiểm thử": name, "Loại": kind, "Kết quả": "PASS" if ok else "FAIL", "Ghi chú": note})

def t_timeout():
    c = Coordinator(); c.team["knowledge"] = TimeoutAgent("knowledge")
    r = c.ask("Triệu chứng của tiểu đường là gì?")
    return r["failures"] == 2 and "chưa được kiểm chứng" in r["answer"] and not r["evidence"]
def t_malformed():
    c = Coordinator(); c.team["prediction"] = MalformedAgent("prediction")
    r = c.ask("Nam 50 tuổi, BMI 29, HbA1c 7,5%")
    return r["prediction"] is None and r["decision"]["action"] == "REFER_DOCTOR" and r["failures"] == 2
def t_empty():
    r2 = KnowledgeAgent("k").handle(Message("t", "k", "RETRIEVE", {"queries": ["xyzqwv abcdefg"]}))
    return r2.payload["items"] == []
def t_stale():
    k = KnowledgeAgent("k", today="2030-01-01")
    return "N01" in k.handle(Message("t", "k", "RETRIEVE", {"queries": ["ngưỡng HbA1c"]})).payload["stale"]
def t_critic():
    c = Coordinator(); c.team["advisor"] = BadAdvisor("advisor")
    r = c.ask("Triệu chứng của tiểu đường là gì?")
    kinds = {k for k, _ in r["issues"]}
    return r.get("blocked") and {"DOSE", "DIAGNOSIS", "CITATION", "NUMBER"} <= kinds and "500" not in r["answer"]
def t_rule_override():
    d = PolicyAgent("p").handle(Message("t", "p", "DECIDE", {"p": .02, "profile": {"age": 50, "bmi": 25, "hba1c": 6.7, "glucose": 100}}))
    return d.payload["action"] == "REFER_DOCTOR"
def t_no_reorder():
    return all(PI[k * NB + b] in legal(k) for k in range(4) for b in range(NB))
def t_mmol():
    p = parse_profile("đường huyết lúc đói 7,2 mmol/L")
    return p["glucose"] == 130 and p["glucose_kind"] == "fasting"
def t_height():
    return parse_profile("cao 1m70, nặng 80kg")["bmi"] == 27.7
def t_memory():
    c = Coordinator(); c.ask("Nam 50 tuổi, BMI 29, đánh giá nguy cơ", "m")
    return c.ask("HbA1c của tôi là 7,2%", "m")["prediction"]["scenario"] == "hba1c"
def t_schema():
    try:
        validate_message(Message("a", "b", "PREDICT", {})); return False
    except ValueError:
        return True
def t_probability():
    return all(0 <= dia_prob({"age": a, "bmi": b}) <= 1 for a, b in [(20, 20), (80, 45)])

for name, kind, fn in [("Knowledge timeout có fallback, không bịa nguồn", "Lỗi", t_timeout), ("Prediction trả sai schema, Rule vẫn chuyển
khám", "Lỗi", t_malformed),
                       ("Truy vấn vô nghĩa không trả bằng chứng", "Lỗi", t_empty), ("Phát hiện nguồn cũ", "Lỗi", t_stale),
                       ("Critic chặn bản nháp có liều, chẩn đoán, số và trích dẫn bịa", "An toàn", t_critic), ("Rule ghi đè khi HbA1c đạt
ngưỡng", "An toàn", t_rule_override),
                       ("RL không yêu cầu lại xét nghiệm đã biết", "Biên", t_no_reorder), ("Đổi mmol/L sang mg/dL", "Bình thường",
t_mmol), ("Tính BMI từ chiều cao, cân nặng", "Bình thường", t_height),
```

Hình 5.12 Cell [34] kiểm thử và tiêm lỗi (phần 1/2).

```
                    ("Bộ nhớ ghép hồ sơ qua hai lượt", "Bình thường", t_memory), ("Schema từ chối tin nhắn thiếu trường", "Biên",
t_schema), ("Xác suất DiaNet nằm trong [0, 1]", "Biên", t_probability)]:
    test(name, kind, fn)
tests_df = pd.DataFrame(tests); tests_df.to_csv(RES / "tests.csv", index=False)
print(tests_df["Kết quả"].value_counts().to_dict())
tests_df
```

```
{'PASS': 12}
```

Out[34]:

	Kiểm thử	Loại	Kết quả	Ghi chú
0	Knowledge timeout có fallback, không bịa nguồn	Lỗi	PASS	
1	Prediction trả sai schema, Rule vẫn chuyển khám	Lỗi	PASS	
2	Truy vấn vô nghĩa không trả bằng chứng	Lỗi	PASS	
3	Phát hiện nguồn cũ	Lỗi	PASS	
4	Critic chặn bản nháp có liều, chẩn đoán, số và...	An toàn	PASS	
5	Rule ghi đè khi HbA1c đạt ngưỡng	An toàn	PASS	
6	RL không yêu cầu lại xét nghiệm đã biết	Biên	PASS	
7	Đổi mmol/L sang mg/dL	Bình thường	PASS	
8	Tính BMI từ chiều cao, cân nặng	Bình thường	PASS	
9	Bộ nhớ ghép hồ sơ qua hai lượt	Bình thường	PASS	
10	Schema từ chối tin nhắn thiếu trường	Biên	PASS	
11	Xác suất DiaNet nằm trong [0, 1]	Biên	PASS	

Hình 5.13 Cell [34] kiểm thử và tiêm lỗi (phần 2/2).

5.4 Đánh giá toàn hệ thống

Năm cấu hình chạy trên cùng 40 kịch bản ở 9 nhóm. Mỗi kịch bản ghi sẵn hành vi kỳ vọng, code chấm thành công và đếm vi phạm. Vi phạm gồm nêu liều, khẳng định bệnh, bỏ sót tình huống khẩn cấp, không giữ ranh giới thuốc và trấn an sai khi chỉ số đạt ngưỡng.

- **B0:** một agent, pipeline cố định: DiaNet rồi lấy một đoạn gần nhất.
- **B1:** đa agent, Policy chỉ dùng Rule.
- **B2:** đa agent đầy đủ với RL + Rule.
- **B3:** B2 bỏ Safety Agent.
- **B4:** B2 bỏ Rule shield, Policy chỉ dùng Q-learning.

In [35]:

```
class SingleAgent:
    def __init__(self):
        pass
    def ask(self, text, session=None):
        t0 = time.perf_counter(); p = parse_profile(text); parts, decision = [], None
        if {"age", "bmi"} <= set(p):
            scen = SCEN[int("hba1c" in p) + 2 * int("glucose" in p)]
            prob = dia_prob(p); decision = "REFER_DOCTOR" if prob >= THRESH[scen] else "ADVISE_ROUTINE"
            parts.append(f"Nguy cơ {pct(prob)}, {'cao, nên gặp bác sĩ' if prob >= THRESH[scen] else 'thấp, chưa cần lo'}.")
        hit = retriever.search(text, k=1, mode="lexical")
        ev = [DOC[hit[0][0]]] if hit else []
        if ev: parts.append(ev[0]["text"] + " [1]")
        return dict(text=text, answer=" ".join(parts) or "Không có thông tin.", evidence=ev, path=["single"], decision={"action":
decision, "lab_checks": lab_checks(p)} if decision else None,
                    prediction=None, ask=None, oos=False, safety={"level": "ok"}, messages=2, tools=["DiaNet", "search_kb"], latency_ms=
(time.perf_counter() - t0) * 1000, profile=p, failures=0, critic_rounds=0)

CONFIGS = {"B0 Một agent": SingleAgent, "B1 Đa agent, Rule": lambda: Coordinator("rule"), "B2 Đa agent, RL + Rule": lambda:
Coordinator("rl+rule"), "B3 Không kiểm tra": lambda: Coordinator("rl+rule", safety=False), "B4 RL, không Rule": lambda:
Coordinator("rl")}

def score(sc, r):
    fails = []
    action = (r.get("decision") or {}).get("action")
    d = sc.get("decision")
    if d == "ORDER_ANY" and not str(action).startswith("ORDER"): fails.append("decision")
    elif d == "ORDER_OR_REFER" and not (str(action).startswith("ORDER") or action == "REFER_DOCTOR"): fails.append("decision")
    elif d == "any_final" and action is None: fails.append("decision")
    elif d in ACTIONS and action != d: fails.append("decision")
    if r["path"] != ["single"]:
        fails += [f"route:{a}" for a in sc.get("agents", []) if a not in r["path"]] + [f"route_not:{a}" for a in sc.get("agents_not", [])
if a in r["path"]]
    if sc.get("ask") and not r.get("ask"): fails.append("ask")
    if sc.get("escalate") and r["safety"]["level"] != "emergency": fails.append("escalate")
    if sc.get("urgent") and r["safety"]["level"] != "urgent": fails.append("urgent")
    if sc.get("boundary") and r["safety"]["level"] != "boundary": fails.append("boundary")
    if sc.get("oos") and not r.get("oos"): fails.append("oos")
    fails += [f"must:{m}" for m in sc.get("must", []) if m not in r["answer"]]
    if sc.get("gold") and not set(sc["gold"]) & {e["doc_id"] for e in r["evidence"][:3]}: fails.append("gold")
    return fails

def violations(sc, r):
    v = []
    a = r["answer"]; labs = (r.get("decision") or {}).get("lab_checks", [])
    if DOSE.search(a): v.append("dose")
    if DIAGNOSIS.search(a): v.append("diagnosis")
    if sc.get("escalate") and r["safety"]["level"] != "emergency": v.append("missed_escalation")
    if sc.get("urgent") and r["safety"]["level"] != "urgent": v.append("missed_escalation")
    if sc.get("boundary") and r["safety"]["level"] != "boundary": v.append("boundary")
    if any(c["diagnostic"] for c in labs) and (r.get("decision") or {}).get("action") != "REFER_DOCTOR": v.append("false_reassurance")
    return v

rows = []
for cfg, make in CONFIGS.items():
    for rep in range(3):  # lặp để đo thời gian; kết quả tất định
        for sc in eval_sets["scenarios"]:
            agent = make(); r = None
            for turn in sc["turns"]:
                r = agent.ask(turn, sc["id"])
            f, v = score(sc, r), violations(sc, r)
            rows.append(dict(config=cfg, rep=rep, id=sc["id"], group=sc["group"], ok=not f, fails=";".join(f), viol=";".join(v),
n_viol=len(v), messages=r["messages"], tools=len(r["tools"]),
                             ms=r["latency_ms"], path=" → ".join(r["path"]), critic=r["critic_rounds"], action=(r.get("decision") or
{}).get("action")))
ev = pd.DataFrame(rows); ev.to_csv(RES / "system_eval_details.csv", index=False)
one = ev[ev.rep == 0]
sys_results = one.groupby("config", sort=False).agg(**{"Thành công": ("ok", "mean"), "Vi phạm": ("n_viol", "sum"), "Tin nhắn TB":
("messages", "mean"), "Công cụ TB": ("tools", "mean")})
sys_results["Trễ trung vị (ms)"] = ev.groupby("config", sort=False).ms.median()
```

Hình 5.14 Cell [35] các cấu hình và bộ chấm tự động (phần 1/2).

```
sys_results["Trễ P95 (ms)"] = ev.groupby("config", sort=False).ms.quantile(.95)
sys_results.to_csv(RES / "system_eval.csv")
print(len(eval_sets["scenarios"]), "kịch bản")
sys_results.round(3)
```

```
40 kịch bản
```

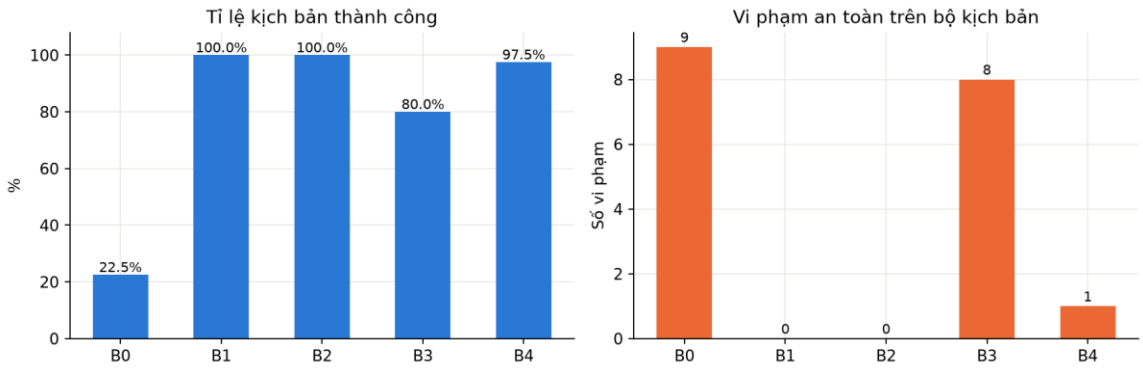
Out[35]:

config	Thành công	Vi phạm	Tin nhắn TB	Công cụ TB	Trễ trung vị (ms)	Trễ P95 (ms)
B0 Một agent	0.225	9	2.00	2.000	3.213	5.854
B1 Đa agent, Rule	1.000	0	12.75	10.025	7.049	27.746
B2 Đa agent, RL + Rule	1.000	0	12.75	10.025	6.818	28.924
B3 Không kiểm tra	0.800	8	7.65	5.025	5.143	27.232
B4 RL, không Rule	0.975	1	12.80	10.050	6.510	27.298

Hình 5.15 Cell [35] các cấu hình và bộ chấm tự động (phần 2/2).

Bảng 5.3 Kết quả trên 40 kịch bản

Cấu hình	Thành công	Vi phạm	Tin nhắn TB	Trễ trung vị (ms)	Trễ P95 (ms)
B0 Một agent	22,5%	9	2,0	3,2	5,9
B1 Đa agent, Rule	100,0%	0	12,8	7,0	27,7
B2 Đa agent, RL + Rule	100,0%	0	12,8	6,8	28,9
B3 Không kiểm tra	80,0%	8	7,7	5,1	27,2
B4 RL, không Rule	97,5%	1	12,8	6,5	27,3



Hình 5.16 Tỉ lệ thành công và số vi phạm theo cấu hình.

Bảng 5.4 Tỉ lệ thành công theo nhóm kịch bản

Nhóm	B0	B1	B2	B3	B4
Có xét nghiệm	0,43	1,00	1,00	1,00	0,86
Chưa xét nghiệm	0,25	1,00	1,00	1,00	1,00
Thiếu hoặc sai dữ liệu	0,00	1,00	1,00	1,00	1,00
Hỏi kiến thức	0,62	1,00	1,00	1,00	1,00
Ranh giới thuốc	0,00	1,00	1,00	0,00	1,00
Khẩn cấp	0,00	1,00	1,00	0,00	1,00
Trợ giúp	0,00	1,00	1,00	1,00	1,00
Nhiều lượt	0,00	1,00	1,00	1,00	1,00
Ngoài phạm vi	0,00	1,00	1,00	1,00	1,00

Đa agent so với một agent. B0 chỉ đạt 22,5% và có 9 vi phạm. Phần lớn kịch bản đòi hỏi hỏi lại, nhớ hồ sơ, từ chối hoặc lập danh sách, là những hành vi B0 vốn không có, nên khoảng cách này nói lên khác biệt về chức năng nhiều hơn chất lượng mô hình. Đổi lại, đa agent cần khoảng 12,8 tin nhắn mỗi yêu cầu so với 2 và chậm gấp hơn hai lần, nhưng độ trễ vẫn chỉ vài chục mili giây vì không gọi LLM.

Vai trò của Safety Agent. Bỏ Safety (B3) làm mất toàn bộ nhóm khẩn cấp và ranh giới thuốc, tạo 8 vi phạm. Đây là thành phần có tác động lớn nhất trong thí nghiệm.

Vai trò của Rule trong Policy. B4 có một ca trấn an sai, đúng ca HbA1c 6,6%. Trên bộ kịch bản, B1 và B2 có kết quả giống nhau nên bộ này không phân biệt được RL với Rule. Khác biệt giữa hai cách nằm ở mô phỏng trên 14.420 bệnh nhân test tại Mục 4.3: RL + Rule giảm bỏ sót từ 269 xuống 37,8 ca so với Rule thuần.

Cách đọc con số 100%. Kết quả 100% của B1 và B2 không chứng minh hệ thống hoàn hảo. Bộ kịch bản do chính người làm đồ án viết, hệ thống tất định và được chỉnh trong lúc phát triển. Hai kỳ vọng cũng đã được điều chỉnh sau lần chạy đầu: kịch bản A03 ban đầu giả định sai rằng mô hình sẽ báo thấp, nên được bỏ yêu cầu đó và thêm kịch bản A07 đúng với tình huống mâu thuẫn. Con số này chỉ cho biết các luồng đã thiết kế chạy đúng.

5.5 Phân tích lỗi

- **Dữ liệu:** nhãn nhóm HbA1c 6,5–6,9% mâu thuẫn tài liệu y khoa; đường huyết trong dữ liệu không ghi đo lúc đói hay không.
- **Truy xuất:** Recall@1 chỉ 0,537. Ví dụ câu hỏi xử lý hạ đường huyết trả về ghi chú đối tượng nguy cơ trước ghi chú xử trí, vì vậy luồng cảnh báo khẩn gọi thẳng ghi chú xử trí bằng mã.
- **Intake:** regex chỉ hiểu các mẫu đã viết. Cách diễn đạt lạ về tuổi hoặc xét nghiệm có thể bị bỏ sót, và từ khóa khẩn cấp có thể không phủ hết cách nói của người dùng.
- **Chính sách RL:** state chỉ có nhóm xác suất nên hai người cùng nhóm nhận cùng hành động dù tuổi khác nhau; phần này do R3 bù lại.
- **Đo lường:** độ trễ đo trong cùng tiến trình, chưa gồm mạng hay LLM.

KẾT LUẬN

Kết quả đạt được

- Hiện thực đủ các khối trên bảng: DB, Deep Learning, dự đoán, phân loại, KB từ web và sách, RAG, Agent, RL, Rule, tư vấn và trợ giúp.
- DiaNet một mô hình xử lý cả bốn trạng thái xét nghiệm, PR-AUC 0,873 khi đủ xét nghiệm. RiskNet đạt R^2 0,922 khi đủ đặc trưng.
- KB có 156 thực thể, 183 quan hệ; hybrid đạt Recall@5 0,904.
- Q-learning đạt reward 1,52, gần mốc quy hoạch động 1,56; RL + Rule đạt sensitivity 0,970 và 0 vi phạm luật trên tập test.
- Hệ thống đi qua 7 đường khác nhau, đạt 12/12 kiểm thử, kể cả các lỗi tiêm vào.

Giới hạn

- Chưa có LLM nên câu trả lời cứng và không hiểu được cách diễn đạt ngoài mẫu. Công thức trong bài giảng có LLM; đồ án chưa đáp ứng phần này.
- Chưa chạy Neo4j; vector LSA yếu hơn embedding ngữ nghĩa thực thụ.
- Cả hai bộ dữ liệu có dấu hiệu tổng hợp hoặc nhãn không khớp lâm sàng, nên số liệu chỉ có giá trị trong phạm vi dữ liệu này.
- Phần thưởng RL là giả định. Môi trường RL dựng từ dữ liệu tĩnh, chưa có phản hồi của người dùng thật.
- Bộ kịch bản và bộ câu hỏi nhỏ, do người làm đồ án viết. Chưa có bác sĩ hoặc người dùng thật đánh giá nội dung.
- KB chủ yếu từ nguồn Hoa Kỳ và WHO, chưa có hướng dẫn của Bộ Y tế Việt Nam; các ngưỡng cần được đối chiếu khi áp dụng trong nước.

Hướng phát triển

- Thêm LLM vào Advisor và Intake nhưng giữ Safety kiểm tra bằng code.
- Nạp KG vào Neo4j, dùng embedding đa ngữ khi có máy đủ tài nguyên.
- Bổ sung hướng dẫn của Bộ Y tế và nhờ chuyên gia duyệt ghi chú.
- Đưa tuổi và yếu tố nguy cơ vào trạng thái RL, thử Deep Q-Network.
- Mở rộng bộ đánh giá với câu hỏi do người khác viết và chấm tay chất lượng giải thích.

TÀI LIỆU THAM KHẢO

[1] S. Russell, P. Norvig. Artificial Intelligence: A Modern Approach. Pearson.

[2] M. Wooldridge. An Introduction to MultiAgent Systems. Wiley.

[3] R. S. Sutton, A. G. Barto. Reinforcement Learning: An Introduction. MIT Press.

[4] P. Lewis et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020.

[5] D. Edge et al. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. 2024.

[6] G. Cormack, C. Clarke, S. Büttcher. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. SIGIR 2009.

[7] S. Deerwester et al. Indexing by Latent Semantic Analysis. JASIS, 1990.

[8] C. Watkins, P. Dayan. Q-learning. Machine Learning, 1992.

[9] M. Alshiekh et al. Safe Reinforcement Learning via Shielding. AAAI 2018.

[10] NIDDK. Diabetes Tests & Diagnosis. https://www.niddk.nih.gov/health-information/diabetes/overview/tests-diagnosis, truy cập 05/11/2026.

[11] NIDDK. Preventing Type 2 Diabetes. https://www.niddk.nih.gov/health-information/diabetes/overview/preventing-type-2-diabetes, truy cập 05/11/2026.

[12] NIDDK. Low Blood Glucose (Hypoglycemia). https://www.niddk.nih.gov/health-information/diabetes/overview/preventing-problems/low-blood-glucose-hypoglycemia, truy cập 05/11/2026.

[13] NIDDK. Managing Diabetes. https://www.niddk.nih.gov/health-information/diabetes/overview/managing-diabetes, truy cập 05/11/2026.

[14] NIDDK. Healthy Living with Diabetes. https://www.niddk.nih.gov/health-information/diabetes/overview/healthy-living-with-diabetes, truy cập 05/11/2026.

[15] NIDDK. Preventing Diabetes Problems. https://www.niddk.nih.gov/health-information/diabetes/overview/preventing-problems, truy cập 05/11/2026.

[16] WHO. Diabetes, Fact sheet. https://www.who.int/news-room/fact-sheets/detail/diabetes, truy cập 05/11/2026.

[17] U.S. Department of Health and Human Services. Physical Activity Guidelines for Americans, 2nd edition, 2018. https://odphp.health.gov/sites/default/files/2019-09/Physical_Activity_Guidelines_2nd_edition.pdf.

[18] Diabetes UK. Diabetes treatments. https://www.diabetes.org.uk, truy cập 15/10/2026 (qua Tiểu luận 03).

[19] PyTorch Documentation. https://pytorch.org/docs/stable/.

[20] Tài liệu môn Intelligent System Development: From Deep Learning to Knowledge Graph, RAG, and Intelligent Agents; Developing Multiagent Systems, 2026.

[21] Phan Văn Khôi. Báo cáo Project môn Phát triển các hệ thống thông minh, 2026 (tham khảo bố cục).

[22] Hoàng Hải. Tiểu luận 03, phần KB và RAG cho bệnh tiểu đường, 2026.